

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



Le Xuan An

**Gradient-Routed Alignment
with Capacity-aware Experts
for Multimodal Understanding**

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Khoa học máy tính

HÀ NỘI - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



LE XUAN AN

**Gradient-Routed Alignment
with Capacity-aware Experts
for Multimodal Understanding**

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Khoa học máy tính

Giảng viên hướng dẫn: Tiến sĩ Nguyễn Văn Sơn

Giảng viên đồng hướng dẫn: Giáo sư Võ Đình Hiếu

HÀ NỘI - 2026

VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY



Le Xuan An

**Gradient-Routed Alignment
with Capacity-aware Experts
for Multimodal Understanding**

BACHELOR'S THESIS

Major: Computer Science

Advisor: Dr. Nguyen Van Son

Co-Supervisor: Professor Vo Dinh Hieu

HANOI - 2026

Lời cảm ơn

Assurance

I hereby affirm that this thesis is my own original work and has not been submitted previously for any academic or professional award. The thesis was completed through my efforts, with important guidance and support from Dr. Nguyen Van Son and Professor Vo Dinh Hieu, for graduation purposes. I confirm that all information, data and findings presented are original and free from plagiarism. Every source consulted has been properly acknowledged and included in the references. I accept full responsibility for the accuracy and integrity of this thesis in accordance with the regulations of VNU University of Engineering and Technology.

Hanoi, Ngày 9 tháng 4 năm 2026

Student

Le Xuan An

Tóm tắt

Tóm tắt nội dung KL tốt nghiệp: Bài toán, phương pháp giải quyết, kết quả đạt được

Từ khóa: Liệt kê 3-5 từ khoá liên quan

Abstract

Summary of the graduation thesis: The problem statement, the proposed approach and the achieved results.

Keywords: Liệt kê 3-5 từ khoá liên quan

Table of Contents

Lời cảm ơn

Assurance i

Tóm tắt ii

Abstract iii

Table of Contents iv

List of Figures vii

List of Tables ix

List of Abbreviations x

Chapter 1 Introduction 1

1.1 Background and Motivation 1

1.2 Problem Statement and Research Objectives 6

1.3 Thesis Contributions 7

1.4 Thesis Outline 8

Chapter 2 Preliminaries 9

2.1 Transformer-Based Vision-Language Models 9

2.1.1 Transformer Basics 9

2.1.2 Vision Encoder and Multimodal Fusion 9

2.1.3 Text-Rich Multimodal Understanding 10

2.2 Knowledge Distillation 10

2.2.1 Teacher-Student Learning 10

2.2.2 Temperature-Scaled Output Distributions 12

2.3 Universal Logit Distillation 12

2.3.1 Motivation 12

2.3.2	ULD Formulation	13
2.3.3	Optimal Transport Perspective	13
2.3.4	Answer-Token Distillation	13
2.4	Multi-Teacher Distillation	15
2.4.1	Per-Teacher KD Loss	15
2.4.2	Learned Routing and Load Balancing	15
2.4.3	Gradient Alignment as a Routing Signal	16
Chapter 3 Proposed Method		19
3.1	Method Overview and Problem Formulation	19
3.1.1	Overall Training Flow	21
3.2	Token-Level Distillation with ULD	21
3.2.1	Supervised Answer Tokens	21
3.2.2	Universal Logit Distillation	22
3.3	Learned Teacher Gate and Load Balancing	22
3.3.1	Student-Conditioned Router	22
3.3.2	Top-k, Capacity, and Fallback	23
3.3.3	Balancing Loss and Expert Bias Update	24
3.4	Gradient Alignment Filtering	26
3.4.1	Gradient Proxies	26
3.4.2	Alignment Score	26
3.4.3	Warmup, Thresholding, and Fallback	26
3.4.4	Sample-Wise KD Aggregation	27
3.5	Final Objective	29
Chapter 4 Evaluation Methodology		30
4.1	Experimental Setup	30
4.2	Datasets	30
4.2.1	Dataset Descriptions	31
4.2.2	Dataset Size and Splits	31
4.3	Training Configuration	31
4.3.1	Offline Teacher-Logit Preparation	33
4.3.2	Remote Logit Streaming During Training	34
4.4	Evaluation Metrics	34
4.5	Baselines and Compared Methods	36
Chapter 5 Experiment Results		39
5.1	Main Comparison Results	39

5.1.1	Baseline Results	39
5.1.2	Overall Comparison	40
5.2	Representation Similarity Analysis	40
5.3	Ablation Study	44
5.4	Qualitative Analysis	44
5.5	Discussion	44
Conclusion and Future Work		47
Chapter A PyTorch-Style GRACE Reference		48
A.1	Core Gate	48
A.2	Core Training Step	48
References		51

List of Figures

1.1	CLIP-style vision-language pretraining aligns image and text representations in a shared embedding space [1, 2].	1
1.2	Scaling-law trends from Kaplan et al.	2
1.3	SmolVLM memory comparison from the Hugging Face release.	2
1.4	Representative compact VLM scales.	3
1.5	TinyLLaVA vs. LLaVA-1.5-7B benchmark comparison.	3
1.6	Best proprietary vs. open-source frontier results on three benchmarks.	4
1.7	Large accuracy drop from naive low-bit VLM quantization.	5
1.8	Teacher-student knowledge distillation.	6
2.1	Transformer attention block structure [3, 4].	9
2.2	Vision Transformer pipeline with patch embedding and Transformer encoder blocks [5, 6].	10
2.3	Representative architecture families of modern vision-language models.	11
2.4	Representative inputs from the text-rich multimodal benchmarks considered in this thesis [7–9].	11
2.5	Generic teacher-student knowledge distillation setup with hard labels and soft targets [10, 11].	12
2.6	Discrete transport plan matrix in optimal transport, showing how source and target marginals are coupled by a transport matrix [12, 13].	14
2.7	ULD compares ranked probability mass after temperature scaling, sorting, and zero-padding to a common vocabulary length [14].	14
2.8	Sparse routing with expert capacity constraints.	16
2.9	Sparse routing and expert specialization in MoE.	17
3.1	Overview of GRACE.	20
3.2	ULD behavior in GRACE.	23
3.3	Gate, sparsification, and optional balance update in GRACE.	24
3.4	Bias-feedback behavior in GRACE.	25
3.5	Routing dynamics in GRACE.	25
3.6	Warmup schedule in GRACE.	27

3.7	Gradient-alignment intuition in GRACE.	28
3.8	Alignment cases in GRACE.	28
3.9	Weight refinement in GRACE.	29
4.1	Example image for a TextVQA-style question [15].	31
4.2	Example document image for a DocVQA-style question [16].	32
4.3	Example chart image for a ChartQA-style question [17].	32
4.4	Training with remote teacher logits. Offline caches are uploaded to R2, fetched lazily during training, and kept in a bounded local cache with oldest- file eviction.	34
5.1	Final-layer CKA heatmaps across two benchmarks.	44
5.2	Vision-layer CKA heatmaps across two benchmarks.	45
5.3	High CKA does not by itself imply that layer-wise transfer will close the downstream performance gap. Representation similarity and task transfer are related, but not equivalent.	46

List of Tables

4.1	Dataset split sizes for the three benchmarks.	33
4.2	Training configuration for the current main distillation script.	35
4.3	Baseline groups used in the evaluation chapter.	36
5.1	Original benchmark results of the two SmolVLM models evaluated with VLM-Eval-Kit.	39
5.2	Original benchmark results of the teacher models used in the heterogeneous teacher pool, compiled from the official model releases [18, 19] and [20, 21] and, for the Qwen2.5-VL-3B ChartQA entry, from a model-comparison page that reports the base-model metric [22].	39
5.3	TextVQA comparison across original models, fine-tuning, distillation base- lines, and the proposed method.	41
5.4	DocVQA comparison across original models, fine-tuning, distillation base- lines, and the proposed method.	42
5.5	ChartQA comparison across original models, fine-tuning, distillation base- lines, and the proposed method.	43

List of Abbreviations

Abbr.	Full Term	Description
ANLS	Average Normalized Levenshtein Similarity	Evaluation metric used for DocVQA.
API	Application Programming Interface	Interface used to query model outputs in black-box settings.
bf16	Brain Floating Point 16	Low-precision floating-point format used for efficient mixed-precision training.
BLIP-2	Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models	A modular vision-language model with a Q-Former bridge.
CCA	Canonical Correlation Analysis	Classical similarity analysis method related to the interpretation of CKA scores.
CE	Cross-Entropy	Supervised loss used for classification or distillation objectives.
CKA	Centered Kernel Alignment	Representation similarity measure used to compare hidden layers between teacher and student models.
ChartQA	Chart Question Answering	Benchmark for question answering over charts.
CLIP	Contrastive Language-Image Pre-training	Dual-encoder vision-language pretraining framework.
CNN	Convolutional Neural Network	Standard visual backbone architecture used as a comparison point to ViT.
DocVQA	Document Visual Question Answering	Benchmark for question answering on document images.
EOS	End of Sequence	Special token that marks the end of a generated answer span.

Abbr.	Full Term	Description
FFN	Feedforward Network	Transformer sublayer applied after attention.
fp16	Floating Point 16	Half-precision floating-point format used in mixed-precision training or inference.
GRACE	Gradient-Routed Alignment with Capacity-aware Experts	Proposed multi-teacher distillation method in this thesis.
GPU	Graphics Processing Unit	Parallel hardware accelerator used for training and serving deep learning models.
HSM-CKA	Hidden-State Matching with CKA	Baseline distillation method that aligns teacher and student hidden representations using CKA.
InfoNCE	Information Noise-Contrastive Estimation	Contrastive objective for aligning image and text representations.
JSON	JavaScript Object Notation	Lightweight structured data format used for the LLaVA-style training files in this thesis.
JS	Jensen-Shannon	Symmetric divergence measure between probability distributions.
KD	Knowledge Distillation	Transfer of knowledge from a teacher model to a student model.
KL	Kullback-Leibler	Measure used to compare probability distributions.
KLD	Kullback-Leibler Divergence	Divergence-based objective widely used in distillation.
LLaVA	Large Language and Vision Assistant	Instruction-tuned projector-based vision-language model.
LLM	Large Language Model	Language model with large parameter scale and strong reasoning ability.
LN	Layer Normalization	Normalization method used to stabilize Transformer training.
MLP	Multi-Layer Perceptron	Feedforward projection block used in Transformers and classifiers.

Abbr.	Full Term	Description
MMD	Maximum Mean Discrepancy	Similarity metric used for feature-level alignment.
MoE	Mixture of Experts	Sparse architecture that routes tokens to selected experts.
MSA	Multi-Head Self-Attention	Attention block that models token interactions in Transformers.
MTKD	Multi-Teacher Knowledge Distillation	Distillation setting that uses more than one teacher model.
NLP	Natural Language Processing	Field concerned with computational understanding of language.
NLS	Normalized Levenshtein Similarity	String similarity score used in DocVQA evaluation.
OCR	Optical Character Recognition	Extraction of text from images or scanned documents.
OT	Optimal Transport	Distribution-matching perspective used to motivate tokenizer-agnostic distillation.
PE	Positional Encoding	Representation of token order in Transformer models.
PLN	Parametric Layer Normalization	Layer normalization variant with trainable scale and bias.
Q-Former	Querying Transformer	Bridging module used by BLIP-2 to connect vision and language features.
QA	Question Answering	Task of generating answers to natural language questions.
ReLU	Rectified Linear Unit	Common non-linear activation function.
RMSNorm	Root Mean Square Layer Normalization	Lightweight normalization method based on root mean square statistics.
RoPE	Rotary Positional Embedding	Positional encoding method based on rotational transformations.
TF32	TensorFloat-32	NVIDIA tensor-core numeric format used to accelerate matrix operations during training.

Abbr.	Full Term	Description
TextVQA	Text Visual Question Answering	Benchmark for question answering on natural images containing text.
ULD	Universal Logit Distillation	Tokenizer-agnostic distillation loss based on ranked probability distributions.
ViT	Vision Transformer	Transformer-based model for image representation learning.
VLM	Vision-Language Model	Model that jointly processes visual and textual inputs.
VLM-Eval-Kit	Vision-Language Model Evaluation Kit	Toolkit used in this thesis to run standardized multimodal evaluation.
VNU	Vietnam National University	Parent university system of VNU University of Engineering and Technology.
VQA	Visual Question Answering	Task of answering questions grounded in visual inputs.
ZeRO	Zero Redundancy Optimizer	Memory-optimization technique used by DeepSpeed to reduce optimizer-state replication during large-model training.

Chapter 1

Introduction

1.1 Background and Motivation

Vision-Language Models (VLMs) jointly process images and text for tasks such as image captioning, visual question answering, document understanding and instruction following [2, 23–26].

In this thesis, *multimodal understanding* refers to practical capabilities such as reading text in images, understanding the input question and performing basic reasoning over visual and textual evidence, rather than broad general intelligence [7–9].

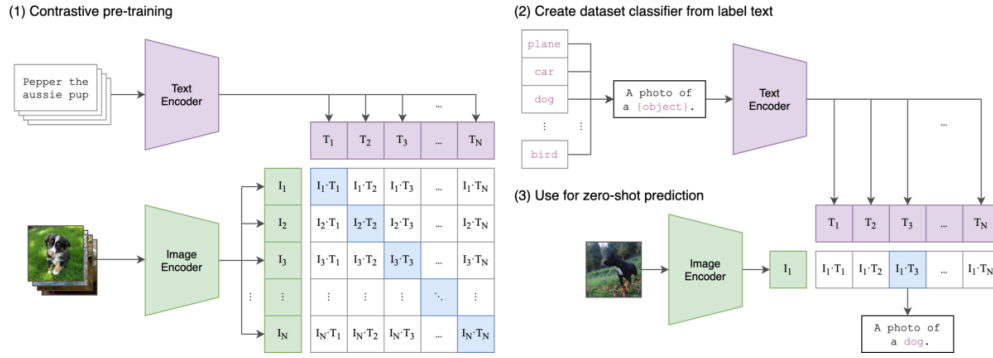


Figure 1.1: CLIP-style vision-language pretraining aligns image and text representations in a shared embedding space [1, 2].

However, high-performing VLMs are often large and expensive in memory, latency, and deployment cost [23, 24, 27, 28]. This limits their use in resource-constrained settings.

This pressure is consistent with scaling-law results. Kaplan et al. show that model quality improves predictably with scale, while Hoffmann et al. show that compute-optimal training still requires jointly increasing model size and data tokens [29, 30]. Larger models therefore remain attractive for accuracy, but they do not become cheap to train or deploy.

The vision side makes this constraint sharper in VLMs. Visual embeddings or visual tokens are inserted into the multimodal context together with text, so the effective sequence length grows with image tokenization density. Since Transformer attention scales quadratically with sequence length, increasing the visual token budget quickly raises memory and latency [4, 25, 31]. For compact VLMs, this means that shrinking only the language backbone is not enough. The visual stack and the number of visual tokens also need to be

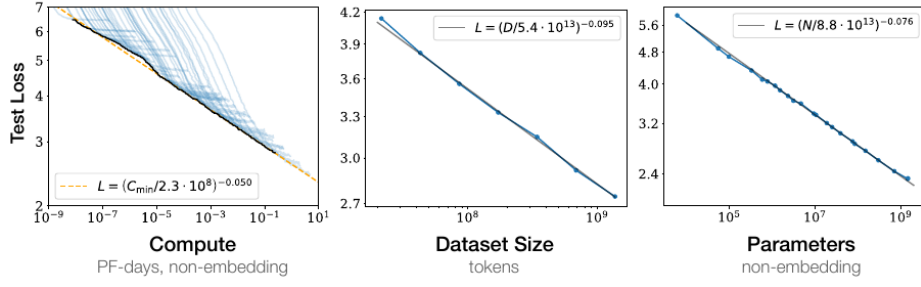


Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Figure 1.2: Scaling-law trends from Figure 1 of Kaplan et al. The reported loss decreases smoothly as compute, dataset size, and parameter count increase, which explains why larger teachers remain attractive even when deployment favors smaller students [29].

controlled.

For this thesis, the most important regime is **small models**, especially **sub-billion VLMs**. Models in the 1B to 3B range are already cheaper than large 7B to 70B systems, but they can still be heavy for edge deployment, memory-limited serving, or repeated evaluation across many checkpoints. Sub-billion students are a stricter target. They force the model to use its parameter budget and visual tokens more carefully, while still aiming to retain useful multimodal performance [27, 28, 31].

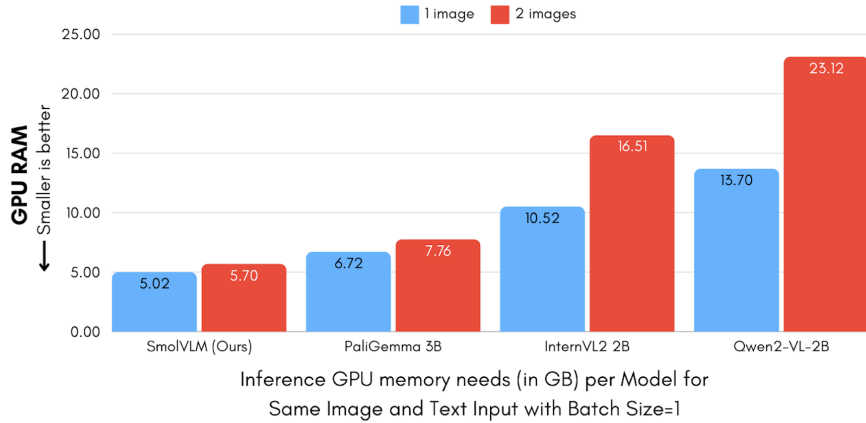


Figure 1.3: Inference GPU memory usage reported in the SmolVLM release blog. The comparison shows that compact VLM design can reduce memory substantially relative to similarly capable models, especially when the visual token budget is controlled [31, 32].

The issue is especially important for text-centric tasks such as scene-text reading, document understanding, and chart reasoning [7–9].

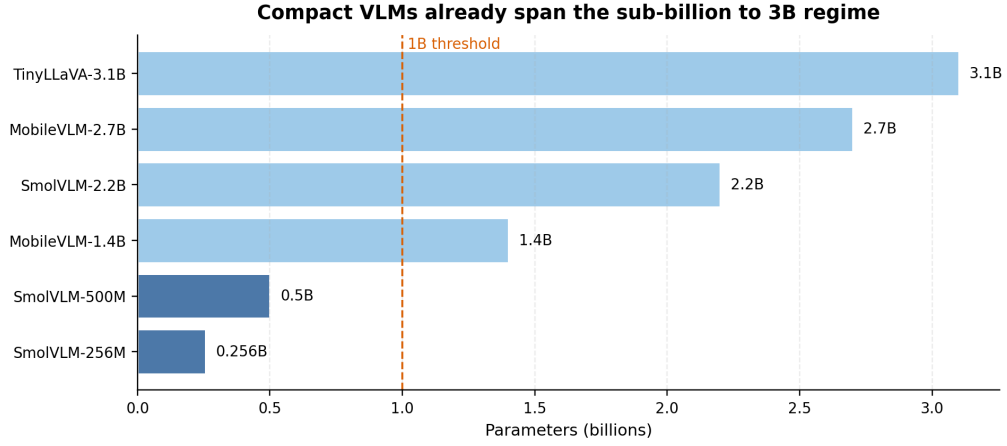


Figure 1.4: Representative compact VLM families already span the sub-billion to 3B regime. This is the design space most relevant to our thesis: the student must stay small enough for efficient deployment, while still benefiting from stronger teachers [27, 28, 31].

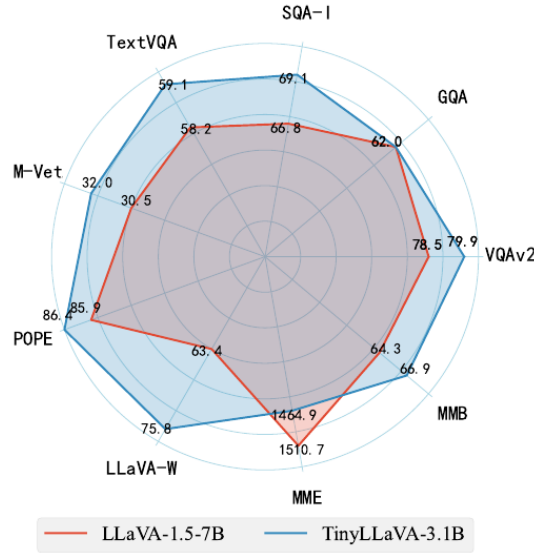


Figure 1.5: Benchmark comparison from TinyLLaVA Figure 1, where a 3.1B small-scale multi-modal model remains competitive with the 7B LLaVA-1.5 baseline across several benchmarks [28].

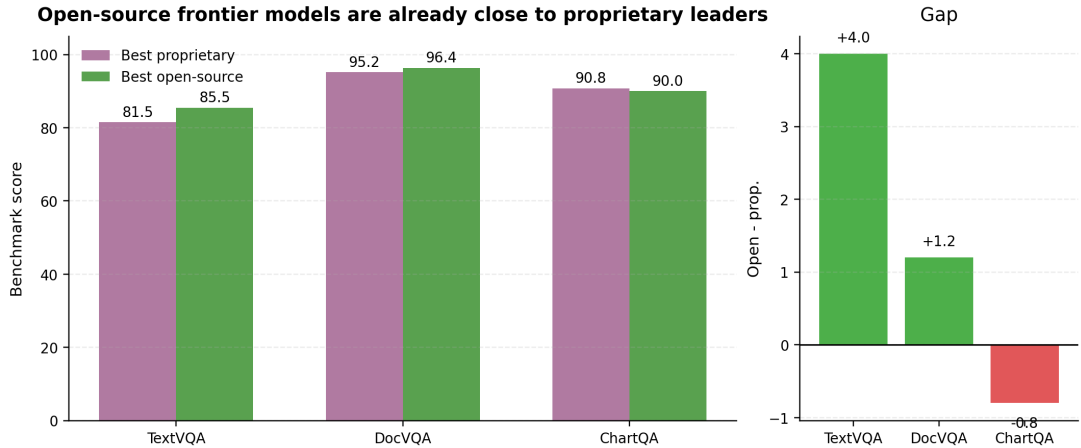


Figure 1.6: Representative proprietary and open-source frontier results on TextVQA, DocVQA, and ChartQA. The gap is already small, and the best open-source model is ahead on TextVQA and DocVQA. The right panel shows the benchmark gap computed as open-source score minus proprietary score from snapshots accessed on April 2, 2026 [33–35].

Knowledge distillation transfers knowledge from a large teacher to a compact student, often preserving strong performance at lower cost [11, 36].

Figure 1.5 shows why this direction matters. Compact multimodal models can already be competitive, but closing the gap to stronger models still requires better transfer of knowledge.

Figure 1.6 sharpens this point. On these three benchmarks, the best open-source models are already close to the best proprietary models, and in two cases they are ahead.

This makes white-box distillation especially attractive. Closed-source systems usually expose only API outputs, while open-source teachers expose logits, tokenizers, and hidden states. That extra access enables objectives such as ULD and hidden-state matching, which are not available in a pure black-box setting [14, 36, 37].

Another major competing compression path is quantization. Unlike distillation, quantization compresses an already trained model by reducing numerical precision. It can cut memory and improve inference speed substantially, so it is a serious practical alternative to KD [38–40].

The trade-off is not free. Low-bit deployment often needs calibration, quantization-aware packing, or custom kernels. In VLMs, the vision and language branches are not equally sensitive to quantization noise, so naive quantization can degrade task accuracy [40, 41].

A stronger published example appears in the MBQ paper. On LLaVA-onevision-7B, the SEED score drops from 71.1 in FP16 to 10.4 under naive W3A16 RTN quantization,

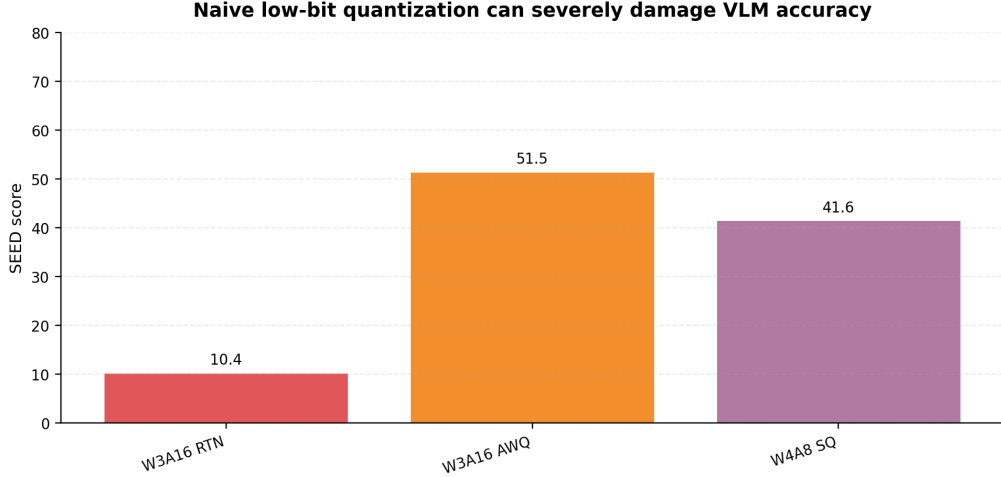


Figure 1.7: SEED accuracy for LLaVA-onevision-7B under naive low-bit quantization baselines, reconstructed from Table 4 of MBQ. The W3A16 RTN baseline drops to 10.4, while other generic low-bit baselines also remain far below the full-precision result reported in the paper [41].

which is a 60.7-point absolute loss. Even stronger VLM-aware quantization methods are motivated by exactly this gap [41].

For this reason, distillation remains attractive in our setting. We want a compact student that is better *before* any later deployment tricks are applied. Quantization can still be combined with the final student, but it does not replace the need for better knowledge transfer during training.

This observation is exactly why we focus on **SmolVLM-500M** and **SmolVLM-256M**. They are small enough to make the efficiency constraint real, yet strong enough that improvements from better distillation remain meaningful [31].

In the multimodal setting, recent work has shown that distillation can transfer reasoning or representation knowledge into more efficient VLMs [26, 42]. This is especially relevant on text-rich benchmarks such as TextVQA, DocVQA, and ChartQA [7–9].

Since no single teacher excels across all multimodal tasks, *multi-teacher distillation* can provide richer supervision than a single-teacher pipeline [43–45].

In our setting, this is not just a matter of picking several strong teachers. The teacher pool contains heterogeneous VLMs with different tokenizers, different output vocabularies, and different sample-level strengths [14, 43, 44].

A naive uniform average can therefore mix teachers that help one sample but hurt another [36, 43, 44].

This motivates a selective multi-teacher framework rather than static teacher averaging.

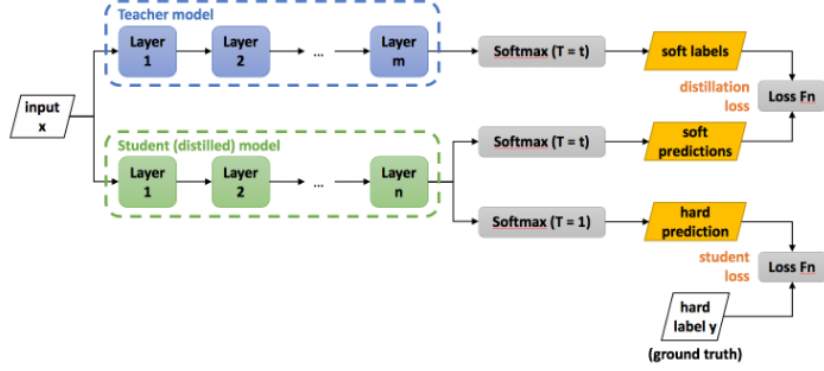


Figure 1.8: Knowledge distillation transfers information from a teacher model to a smaller student model [10, 11].

The concrete problem and design requirements are stated in the next section.

1.2 Problem Statement and Research Objectives

The central problem addressed in this thesis is how to transfer multimodal understanding ability from large VLMs to compact student models without paying the full cost of large-scale deployment.

We focus on text-rich multimodal tasks where the student must read image text, understand the question, and perform basic reasoning [7–9].

In this setting, existing distillation pipelines often assume a single teacher, a shared tokenizer, or a fixed averaging rule across teachers. These assumptions are restrictive for heterogeneous multi-teacher VLM distillation [11, 14, 43, 44].

More specifically, the method should satisfy three requirements.

1. The KD loss should remain valid even when teacher and student tokenizers differ.
2. Teacher assignment should be *sample-dependent* rather than fixed across the whole dataset.
3. The routing mechanism should remain sparse while suppressing routed teachers whose KD direction conflicts with the supervised objective. When needed, the framework should also support regularizers that discourage routing collapse.

Given a pool of N frozen teacher VLMs and a compact student, the goal is therefore to learn a distillation strategy that:

1. remains valid across mismatched teacher and student vocabularies,
2. routes teachers adaptively from student-side features through a learned deep gate,
3. retains sparse top- k computation under capacity constraints,
4. and filters routed teachers when their KD update direction conflicts with the supervised objective, while keeping a fallback teacher when necessary.

This thesis addresses these requirements with Gradient-Routed Alignment with Capacity-aware Experts (GRACE).

GRACE is a selective multi-teacher distillation framework.

It uses tokenizer-agnostic Universal Logit Distillation (ULD), a deep student-conditioned teacher gate, top- k capacity-aware sparse routing, and gradient-alignment filtering with a least-negative fallback. The framework also supports balancing and expert-bias updates as optional routing regularizers [14, 46–48].

We evaluate it on text-rich multimodal understanding tasks such as TextVQA, DocVQA, and ChartQA [7–9].

Accordingly, this thesis has three main objectives:

1. formulate cross-tokenizer multimodal distillation for compact VLM students in a text-centric benchmark setting,
2. design GRACE as a selective multi-teacher distillation framework with ULD, a deep student-conditioned gate, top- k capacity-aware routing, and gradient alignment, while supporting optional balancing and expert-bias updates,
3. evaluate whether GRACE improves transfer for small VLMs on TextVQA, DocVQA and ChartQA, and analyze why simple layer-wise transfer is less effective in this setting.

1.3 Thesis Contributions

The main contributions are:

1. We formulate a practical multi-teacher distillation setting for compact VLMs with heterogeneous teacher vocabularies on text-rich multimodal benchmarks.

2. We propose Gradient-Routed Alignment with Capacity-aware Experts (GRACE), a unified distillation framework for heterogeneous teachers. It combines Universal Logit Distillation, a deep student-conditioned routing gate, top- k capacity-aware sparse assignment, and gradient-alignment filtering, while also supporting optional balancing and expert-bias updates.
3. We provide empirical evidence on TextVQA, DocVQA, and ChartQA that this design yields stronger transfer than standard fine-tuning and conventional logits-based distillation baselines for compact VLM students.
4. We analyze representation similarity and layer-wise transfer, showing why high internal similarity alone is insufficient and why naive layer matching is a weak distillation bias in this setting.
5. We complement the headline comparison with routing and ablation analysis, clarifying how selective teacher assignment contributes to the final gains.

1.4 Thesis Outline

The remainder of this thesis is organized as follows:

1. Chapter 2 reviews the background on Transformers, VLMs, knowledge distillation, Universal Logit Distillation, sparse routing, gradient alignment, and CKA.
2. Chapter 3 presents GRACE, including the tokenizer-agnostic KD objective, deep teacher gate, top- k capacity-aware routing, alignment filtering, and the optional routing regularizers used in the full framework.
3. Chapter 4 describes the experimental setup, datasets, evaluation metrics and compared methods.
4. Chapter 5 reports the baseline results, comparison results, ablation studies and discussion.
5. The conclusion summarizes the main findings, highlights the limitations of the current study, and outlines directions for future work.

Chapter 2

Preliminaries

2.1 Transformer-Based Vision-Language Models

2.1.1 Transformer Basics

Modern vision-language models are built on the Transformer architecture [4]. For an input sequence, self-attention maps query, key and value matrices $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$ to

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}, \quad (2.1)$$

where d_k is the key dimension. Multi-head attention applies this operation in parallel across several learned subspaces and is followed by a feed-forward network and residual normalization.

For language modeling, the Transformer produces a sequence of hidden states and next-token logits. In this thesis, the student and teacher models are trained or evaluated through their output logits because the distillation method operates at the token-distribution level [4, 11].

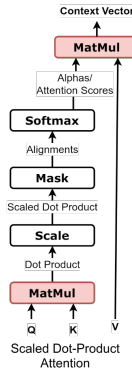


Figure 2.1: Transformer attention block structure [3, 4].

2.1.2 Vision Encoder and Multimodal Fusion

On the visual side, many multimodal systems use a Vision Transformer (ViT) [6], which converts an image into a sequence of patch embeddings and processes them with Transformer blocks. These visual tokens are then fused with text by a connector module such as cross-attention, a Q-Former, or a learned projector [23–25].

This thesis focuses on compact generative VLMs in the same general family as SmolVLM and related projector-based models. Conceptually, such a model can be written as

$$\mathbf{z} = f_{\theta}(x), \quad (2.2)$$

where x is a multimodal input and $\mathbf{z}$ denotes the output logits used for autoregressive answer generation.

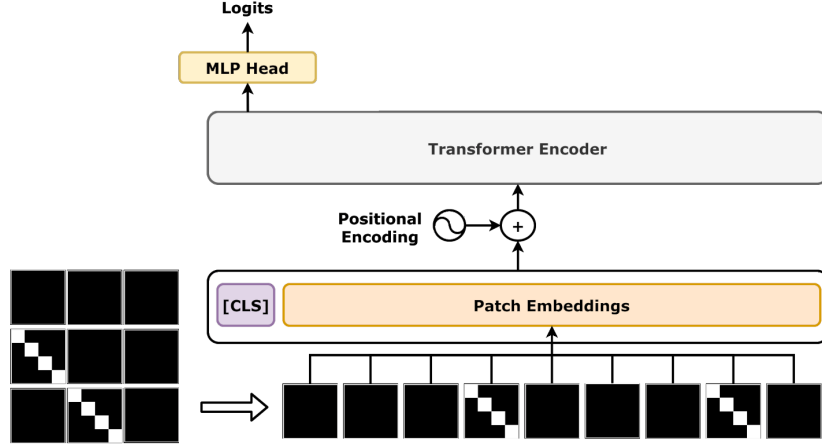


Figure 2.2: Vision Transformer pipeline with patch embedding and Transformer encoder blocks [5, 6].

2.1.3 Text-Rich Multimodal Understanding

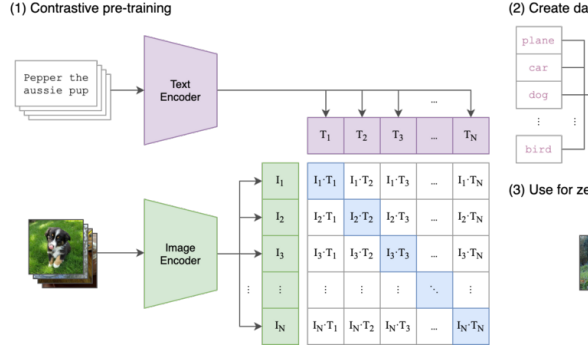
The tasks considered in this thesis, namely TextVQA, DocVQA, and ChartQA, require the model to read text in images, understand the question, and perform basic reasoning over visual and textual evidence [7–9]. This setting differs from generic image captioning because success depends strongly on OCR-sensitive recognition and token-level answer generation.

2.2 Knowledge Distillation

2.2.1 Teacher-Student Learning

Knowledge distillation transfers information from a stronger teacher model to a smaller student model [11, 36]. Let $\mathcal{L}_{CE}$ denote the supervised cross-entropy loss of the student. A generic distillation objective can be written as

$$\mathcal{L} = \lambda_{CE}\mathcal{L}_{CE} + \lambda_{KD}\mathcal{L}_{KD}, \quad (2.3)$$



(a) CLIP-style dual encoder [2].

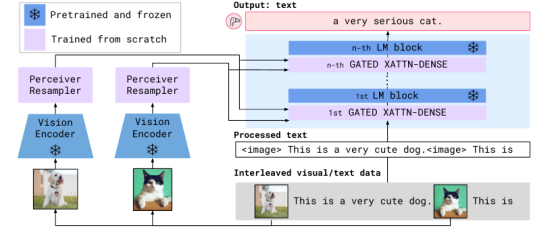
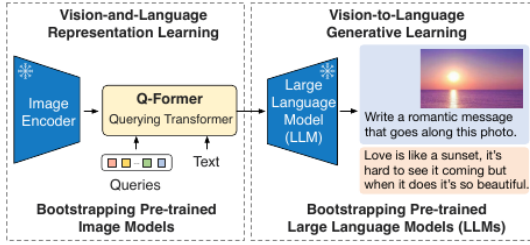


Figure 3: Flamingo architecture overview. Flamingo is a family of visual language models (VLMs) that take as input visual data interleaved with text and produce free-form text as output.

(b) Flamingo cross-attention design [23].



(c) BLIP-2 with Q-Former [24].

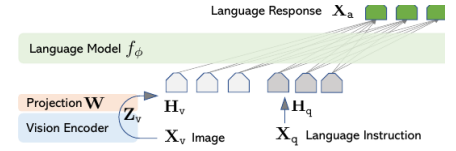


Figure 1: LLaVA network architecture.

(d) LLaVA projector-based design [25].

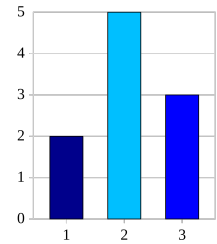
Figure 2.3: Representative architecture families of modern vision-language models.



(a) TextVQA



(b) DocVQA



(c) ChartQA

Figure 2.4: Representative inputs from the text-rich multimodal benchmarks considered in this thesis [7–9].

where $\mathcal{L}_{KD}$ is a teacher-guided loss. In later chapters, GRACE uses the specific convex-mixing form $(1 - \alpha)\mathcal{L}_{CE} + \alpha\mathcal{L}_{KD}$.

In token-level distillation, the teacher and student are compared on their output logits rather than only on hard labels. This allows the student to learn richer distributional structure, often called *dark knowledge* [11].

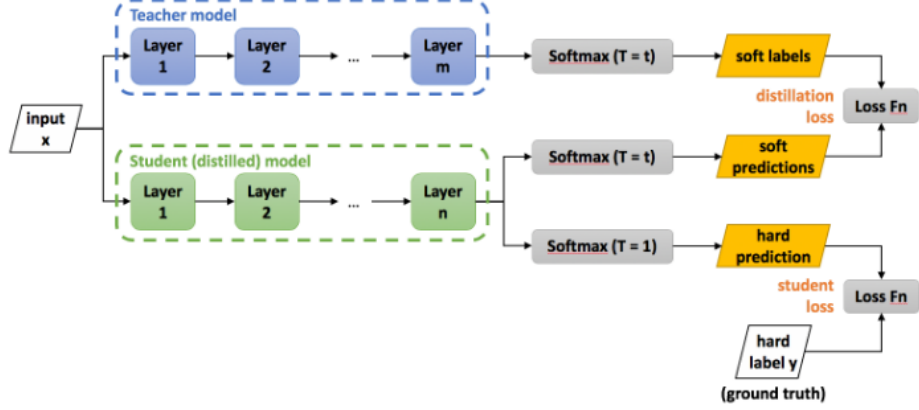


Figure 2.5: Generic teacher-student knowledge distillation setup with hard labels and soft targets [10, 11].

2.2.2 Temperature-Scaled Output Distributions

Let $\mathbf{z}_s$ and $\mathbf{z}_t$ denote student and teacher logits. With temperatures τ_s and τ_t , the corresponding distributions are

$$\mathbf{p}_s = \text{softmax}\left(\frac{\mathbf{z}_s}{\tau_s}\right), \quad \mathbf{p}_t = \text{softmax}\left(\frac{\mathbf{z}_t}{\tau_t}\right). \quad (2.4)$$

Many classical KD methods compare $\mathbf{p}_s$ and $\mathbf{p}_t$ using divergences such as forward KL or Jensen-Shannon divergence. However, such approaches usually assume that teacher and student share the same vocabulary [11, 14, 49].

2.3 Universal Logit Distillation

2.3.1 Motivation

In this thesis, the teacher pool may contain different VLM families with different tokenizers and vocabulary sizes. Therefore, a KD loss that assumes a shared output space is too restrictive. To address this, we use Universal Logit Distillation (ULD) [14].

2.3.2 ULD Formulation

Given student logits $\mathbf{z}_s \in \mathbb{R}^{V_s}$ and teacher logits $\mathbf{z}_t \in \mathbb{R}^{V_t}$, we first compute their temperature-scaled probability distributions $\mathbf{p}_s$ and $\mathbf{p}_t$. We then sort both vectors in descending order, pad the smaller vocabulary with zeros, and compare the sorted masses with an L_1 distance:

$$\mathcal{L}_{ULD}(\mathbf{z}_s, \mathbf{z}_t) = \|\pi(\mathbf{p}_s) - \pi(\mathbf{p}_t)\|_1, \quad (2.5)$$

where $\pi(\cdot)$ denotes sorting in descending order after zero-padding.

The key advantage of ULD is that it compares distributions by their ranked probability mass rather than by token identity. This makes it directly applicable to cross-tokenizer teacher-student distillation [14].

2.3.3 Optimal Transport Perspective

Optimal transport compares two probability measures by the minimum cost required to move mass from one distribution to the other [13]. For discrete source weights $\mathbf{a}$, target weights $\mathbf{b}$, and transport cost matrix $\mathbf{C}$, the corresponding optimization problem can be written as

$$W(\mathbf{a}, \mathbf{b}) = \min_{\mathbf{P} \in \mathcal{U}(\mathbf{a}, \mathbf{b})} \langle \mathbf{P}, \mathbf{C} \rangle, \quad \mathcal{U}(\mathbf{a}, \mathbf{b}) = \{\mathbf{P} \geq 0 \mid \mathbf{P}\mathbf{1} = \mathbf{a}, \mathbf{P}^\top \mathbf{1} = \mathbf{b}\}. \quad (2.6)$$

Here, $\mathbf{P}$ is a transport plan that specifies how much mass is moved from each source bin to each target bin.

This viewpoint helps explain why ULD works across tokenizers.

When teacher and student have different vocabularies, matching probabilities by token index is not meaningful. What matters instead is how much probability mass is assigned to highly ranked outcomes [13, 14].

ULD does not solve a full OT problem at each training step. However, it follows the same geometric view: compare the shape of the mass distribution rather than enforcing one-to-one token correspondence [14].

The sorted-probability surrogate in ULD is therefore an efficient tokenizer-agnostic approximation to mass-matching behavior [13, 14].

2.3.4 Answer-Token Distillation

For multimodal question answering, distillation should focus on answer tokens rather than prompt tokens [25].

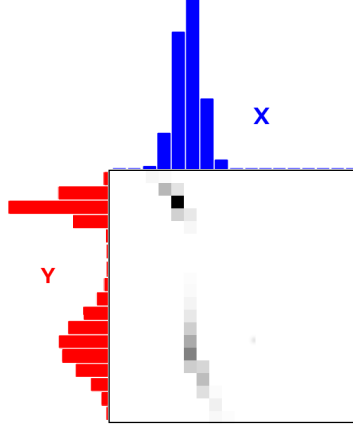


Figure 2.6: Discrete transport plan matrix in optimal transport, showing how source and target marginals are coupled by a transport matrix [12, 13].

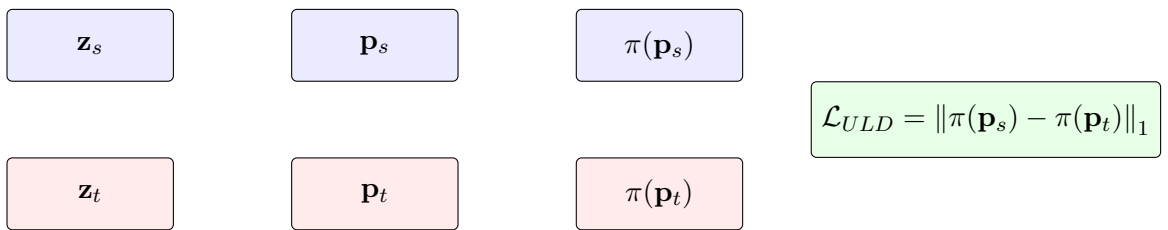


Figure 2.7: ULD compares ranked probability mass after temperature scaling, sorting, and zero-padding to a common vocabulary length [14].

In our implementation, KD is computed only on supervised answer positions after masking ignored tokens, optionally removing EOS, and truncating student and teacher answer spans to a common length.

In Chapter 3, this supervised answer-position set is denoted by Ω_b .

This gives a token-aligned distillation signal while remaining compatible with different tokenizers.

2.4 Multi-Teacher Distillation

2.4.1 Per-Teacher KD Loss

Let $\mathcal{T} = \{T_1, \dots, T_N\}$ be a pool of frozen teachers. For sample b , teacher T_i produces a per-teacher distillation loss $\mathcal{L}_{KD}^{(b,i)}$. In principle, one could average these losses uniformly,

$$\mathcal{L}_{KD}^{(b)} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_{KD}^{(b,i)}, \quad (2.7)$$

but this assumes every teacher is equally useful for every sample.

In practice, different teachers may provide conflicting guidance. Some teachers may help the supervised objective on a given sample, while others may push the student in a different direction [48]. This motivates a selective routing mechanism that decides, for each sample, which teachers should be active.

2.4.2 Learned Routing and Load Balancing

Recent sparse expert models route each sample or token through only a subset of available experts. This improves efficiency and allows specialized computation [46, 50, 51].

The same high-level idea is useful in multi-teacher distillation. Instead of averaging all teachers equally, a learned router can propose teacher weights from student-side features [43–45].

Let $\mathbf{h}^{(b)}$ denote a pooled student representation for sample b . A router network $G(\cdot)$ produces gate logits

$$\mathbf{r}^{(b)} = G(\mathbf{h}^{(b)}), \quad \mathbf{p}^{(b)} = \text{softmax}(\mathbf{r}^{(b)}), \quad (2.8)$$

where $p_i^{(b)}$ is the initial routing weight for teacher T_i .

In sparse routing, these weights are typically refined by additional constraints:

1. **top- k routing**, which keeps only the highest-scoring teachers,
2. **capacity constraints**, which prevent one teacher from receiving too many assignments,
3. **load balancing**, which discourages routing collapse onto a small subset of teachers.

Following the general MoE literature, one can summarize the balancing signal by a soft load $\bar{p}_i$ and a hard assignment load $\bar{q}_i$, and define an auxiliary balancing objective of the form

$$\mathcal{L}_{bal} = N \sum_{i=1}^N \bar{p}_i \bar{q}_i. \quad (2.9)$$

This auxiliary term does not replace the KD objective; instead, it regularizes the router so that different teachers remain usable during training.

In GRACE, the router is implemented as a deep student-conditioned gate on top of pooled student hidden states. It is followed by softmax weights, top- k sparsification, and capacity filtering. The full framework also supports optional balancing and expert-bias updates.

When enabled, the expert bias term nudges the router toward under-used teachers in subsequent batches.

This makes the routing stage closer to sparse-expert training than to a static teacher average [46, 47, 50].

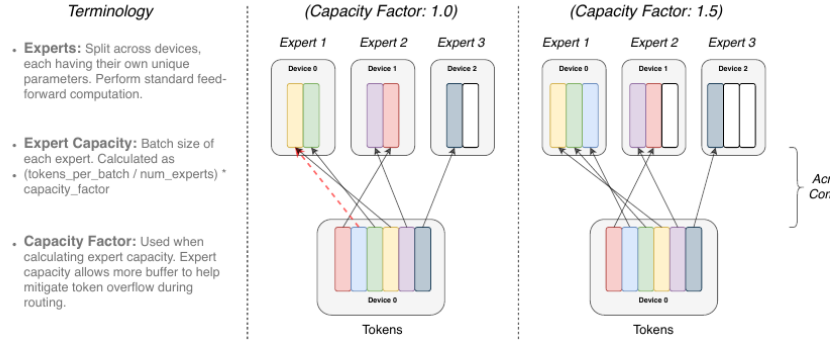


Figure 2.8: Illustration of sparse routing with expert capacity constraints, adapted from Switch Transformers [46]. Capacity factors control how many routed samples each expert can accept before overflow, which is directly analogous to the capacity-aware teacher routing used in GRACE.

2.4.3 Gradient Alignment as a Routing Signal

Gradient similarity has been used in distillation as a signal for deciding whether teacher supervision helps or conflicts with the student objective [48].

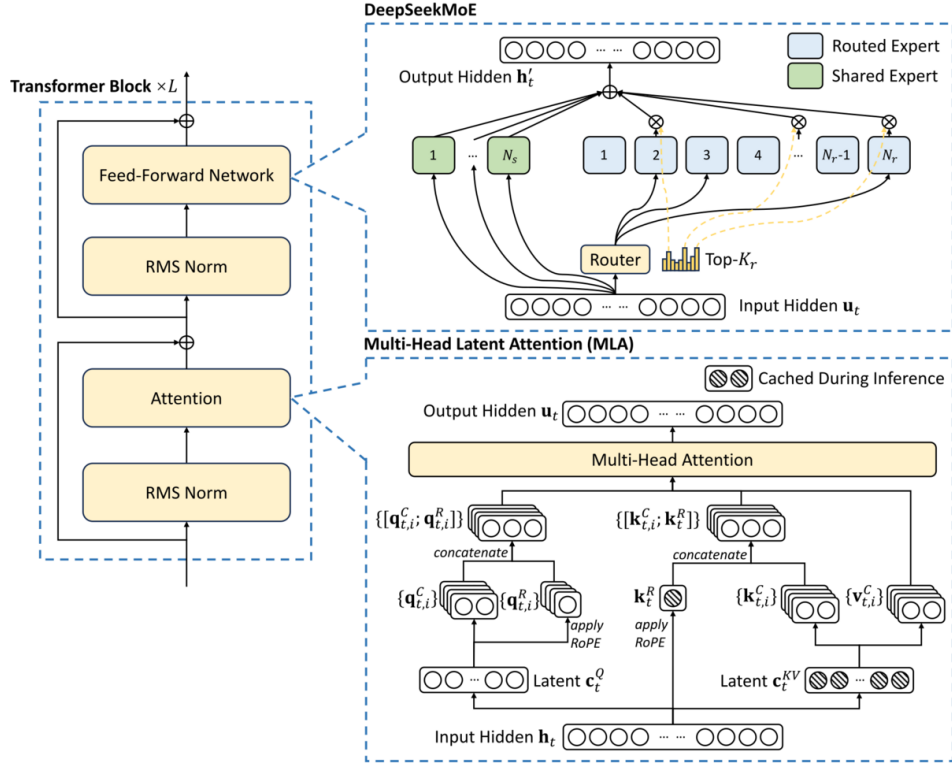


Figure 2.9: Example of sparse routing and expert specialization in a mixture-of-experts architecture [51, 52]. Although our setting is multi-teacher distillation rather than token-level MoE language modeling, the same principles of learned routing, sparse activation, and load balancing are directly relevant.

The high-level idea is simple. If the teacher-induced update points in a similar direction to the supervised update, the teacher is more likely to be helpful. If the two directions conflict, teacher supervision may hurt learning on that sample [48].

In this thesis, GRACE uses this idea only after routing. The GRACE-specific formulation of gradient proxies, cosine alignment, thresholding, and fallback is presented in Chapter 3, where it is part of the method rather than general background.

Chapter 3

Proposed Method

3.1 Method Overview and Problem Formulation

We consider text-rich multimodal understanding tasks such as TextVQA, DocVQA, and ChartQA [7–9]. The training set is

$$\mathcal{D} = \{(x_b, y_b)\}_{b=1}^B, \quad (3.1)$$

where x_b is a multimodal input and y_b is the target answer sequence.

Let S_θ denote the student VLM with trainable parameters θ , and let

$$\mathcal{T} = \{T_1, T_2, \dots, T_N\} \quad (3.2)$$

be a pool of frozen teacher VLMs. For each sample b , the student produces logits $\mathbf{z}_s^{(b)}$, and teacher T_i produces teacher logits $\mathbf{z}_t^{(b,i)}$.

The goal of this thesis is not to search for a fixed teacher subset before training. Instead, the student is trained with *Gradient-Routed Alignment with Capacity-aware Experts (GRACE)*.

GRACE is organized around four interacting mechanisms:

1. a supervised student objective $\mathcal{L}_{CE}$,
2. a per-teacher logit distillation objective $\mathcal{L}_{KD}^{(b,i)}$,
3. a deep teacher gate with top- k sparsification and capacity control,
4. optional balancing and expert-bias regularization for the router,
5. a cosine gradient-alignment filter with thresholding and least-negative fallback applied on top of the routed teachers.

At a high level, each teacher first produces a candidate KD signal for the current sample.

The student-side gate proposes sparse routing weights from pooled hidden features. When balancing is enabled, it regularizes the gate against collapse. Gradient alignment removes routed teachers whose KD direction is not sufficiently consistent with the supervised objective.

If all routed teachers fall below the alignment threshold, the method keeps the least-negative routed teacher as a fallback. The final student update is

$$\mathcal{L} = (1 - \alpha)\mathcal{L}_{CE} + \alpha\mathcal{L}_{KD} + \lambda_{bal}\mathcal{L}_{bal}, \quad (3.3)$$

where $\alpha \in [0, 1]$ controls the KD weight and λ_{bal} controls the optional gate-balancing term.

This design is closer in spirit to selective gradient-based distillation [48] and sparse expert routing [46, 50] than to static teacher averaging.

Here it is adapted to the multi-teacher multimodal setting with token-level logit distillation.

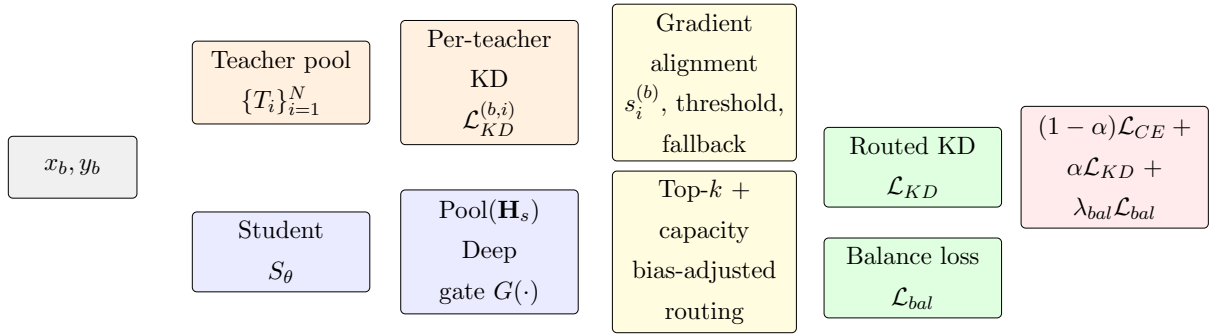


Figure 3.1: Overview of GRACE. The student produces both supervised learning signals and routing features. The gate proposes sparse teacher assignments, capacity control stabilizes routing, and gradient alignment filters routed teachers before KD aggregation and the final update. The full framework can additionally enable balancing and expert-bias updates.

3.1.1 Overall Training Flow

Algorithm 1 Gradient-Routed Alignment with Capacity-aware Experts (GRACE)

Require: mini-batch $\{(x_b, y_b)\}_{b=1}^B$, frozen teachers $\{T_i\}_{i=1}^N$

- 1: Run S_θ to obtain student logits, hidden states, and $\mathcal{L}_{CE}$
 - 2: Pool student hidden states and compute gate scores
 - 3: Apply top- k routing, capacity control, and routing fallback
 - 4: **for** each routed teacher T_i **do**
 - 5: Compute teacher logits and the per-teacher KD loss
 - 6: **end for**
 - 7: Compute CE and KD gradient proxies, then alignment scores
 - 8: **if** warmup is finished **then**
 - 9: Keep routed teachers with alignment score above δ
 - 10: **if** none survive **then**
 - 11: Keep the least-negative routed teacher as fallback
 - 12: **end if**
 - 13: **end if**
 - 14: Aggregate routed KD, add optional balance loss, and update θ
-

Algorithm 1 gives the full training flow before the sections below define each component in detail. In the current main configuration reported in Chapter 4, the key GRACE-specific hyperparameters are $k = 1$, capacity factor $\phi = 4.0$, balance weight $\lambda_{bal} = 0.0$, bias-update rate $\eta_b = 0.0$, alignment warmup ratio $\rho = 0.2$, alignment threshold $\delta = -0.02$, and KD weight $\alpha = 0.5$.

3.2 Token-Level Distillation with ULD

3.2.1 Supervised Answer Tokens

Distillation is applied only on supervised answer tokens, not on prompt tokens. For sample b , let Ω_b denote the set of supervised token positions after:

1. masking ignored labels,
2. optionally removing the EOS token from both student and teacher answer spans,
3. truncating student and teacher answer spans to a common length.

This produces aligned token sequences for the student and each teacher. The alignment is important because teacher and student models may differ in tokenizer and vocabulary size.

3.2.2 Universal Logit Distillation

For each sample b , token position $j \in \Omega_b$, and teacher T_i , let $\mathbf{z}_{s,j}^{(b)} \in \mathbb{R}^{V_s}$ be the student logits and $\mathbf{z}_{t,j}^{(b,i)} \in \mathbb{R}^{V_i}$ be the teacher logits. The student and teacher probability distributions are

$$\mathbf{p}_{s,j}^{(b)} = \text{softmax}\left(\frac{\mathbf{z}_{s,j}^{(b)}}{\tau_s}\right), \quad \mathbf{p}_{t,j}^{(b,i)} = \text{softmax}\left(\frac{\mathbf{z}_{t,j}^{(b,i)}}{\tau_t}\right), \quad (3.4)$$

where τ_s and τ_t are the student and teacher temperatures.

Following Universal Logit Distillation (ULD) [14], we sort both probability vectors in descending order and pad the smaller vocabulary with zeros.

Let $\pi(\cdot)$ denote descending sort after zero-padding to a common size. The token-level ULD loss is

$$\ell_{ULD}(\mathbf{z}_{s,j}^{(b)}, \mathbf{z}_{t,j}^{(b,i)}) = \left\| \pi(\mathbf{p}_{s,j}^{(b)}) - \pi(\mathbf{p}_{t,j}^{(b,i)}) \right\|_1. \quad (3.5)$$

The per-sample, per-teacher KD loss is the mean over supervised answer tokens:

$$\mathcal{L}_{KD}^{(b,i)} = \frac{1}{|\Omega_b|} \sum_{j \in \Omega_b} \ell_{ULD}(\mathbf{z}_{s,j}^{(b)}, \mathbf{z}_{t,j}^{(b,i)}). \quad (3.6)$$

ULD is particularly suitable here because it does not require the student and teacher to share the same vocabulary [14].

This matters in our setting because the teacher pool may contain VLMs from different model families.

3.3 Learned Teacher Gate and Load Balancing

3.3.1 Student-Conditioned Router

When the teacher pool contains multiple models, the method first builds a student-conditioned router instead of averaging teachers uniformly.

Let $\mathbf{H}_s^{(b)}$ denote the student hidden states captured at the routing source layer for sample b . We pool these hidden states over supervised answer positions to obtain a routing feature

$$\mathbf{h}^{(b)} = \text{Pool}(\mathbf{H}_s^{(b)}). \quad (3.7)$$

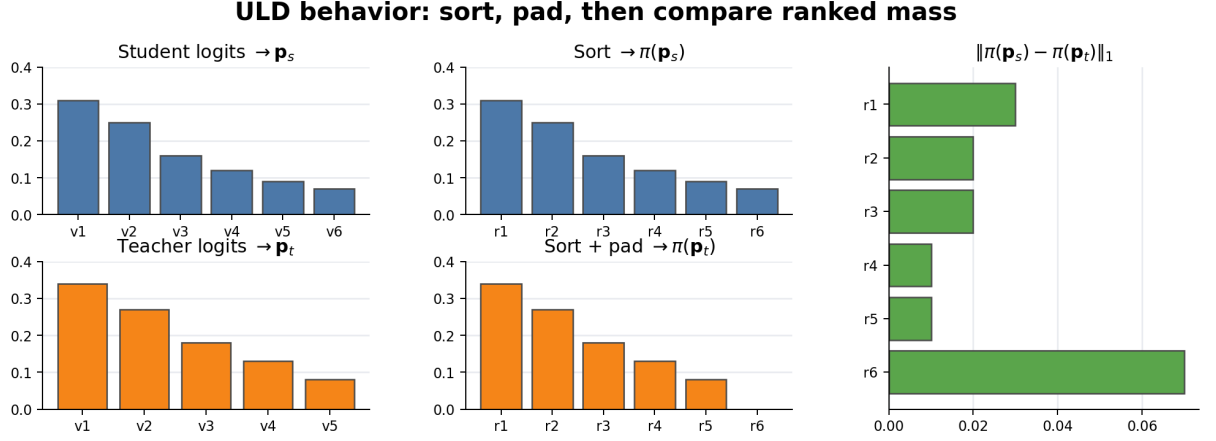


Figure 3.2: ULD behavior in GRACE. Student and teacher probabilities are first softened, then sorted and zero-padded to a common size before the ranked probability masses are compared with an L_1 distance. This avoids direct dependence on shared token identities.

A deep router $G(\cdot)$ then produces gate logits

$$\mathbf{r}^{(b)} = G(\mathbf{h}^{(b)}), \quad \mathbf{p}^{(b)} = \text{softmax}(\mathbf{r}^{(b)}), \quad (3.8)$$

where $p_i^{(b)}$ is the initial gate weight for teacher T_i .

3.3.2 Top-k, Capacity, and Fallback

The router is made sparse by applying top- k routing to bias-adjusted scores

$$\tilde{r}_i^{(b)} = r_i^{(b)} + b_i, \quad (3.9)$$

where b_i is a teacher-specific expert bias. Let $\mathcal{K}_b = \text{TopK}(\tilde{\mathbf{r}}^{(b)}, k)$ be the candidate teacher set for sample b . To prevent one teacher from receiving too many assignments, each teacher is additionally limited by a capacity

$$C = \left\lceil \phi \frac{Bk}{N} \right\rceil, \quad (3.10)$$

where B is the mini-batch size, N is the number of teachers, and $\phi > 0$ is the capacity factor.

Let $m_i^{(b)} \in \{0, 1\}$ denote the resulting assignment mask after top- k selection and capacity pruning. The constrained gate weights are

$$\hat{p}_i^{(b)} = \frac{m_i^{(b)} p_i^{(b)}}{\sum_{j=1}^N m_j^{(b)} p_j^{(b)}}. \quad (3.11)$$

If capacity pruning leaves a sample without any assigned teacher, the algorithm falls back to the teacher with the largest bias-adjusted routing score $\tilde{r}_i^{(b)}$.

This ensures that at least one teacher remains active before alignment filtering.

3.3.3 Balancing Loss and Expert Bias Update

The full GRACE framework can optionally regularize the gate with a balancing loss that uses both soft routing weights and hard assignments.

Define the soft load

$$\bar{p}_i = \frac{1}{B} \sum_{b=1}^B p_i^{(b)} \quad (3.12)$$

and the hard assignment load

$$\bar{q}_i = \frac{1}{Bk} \sum_{b=1}^B \sum_{j=1}^k \mathbf{1}[\text{TopK}_j(\tilde{\mathbf{r}}^{(b)}) = i]. \quad (3.13)$$

The balancing objective is

$$\mathcal{L}_{bal} = N \sum_{i=1}^N \bar{p}_i \bar{q}_i. \quad (3.14)$$

When enabled, the expert bias is updated from the per-batch assignment load ℓ_i to discourage teacher overuse:

$$b_i \leftarrow b_i - \eta_b \left(\ell_i - \frac{1}{N} \sum_{j=1}^N \ell_j \right), \quad (3.15)$$

where η_b is a small bias-update rate.

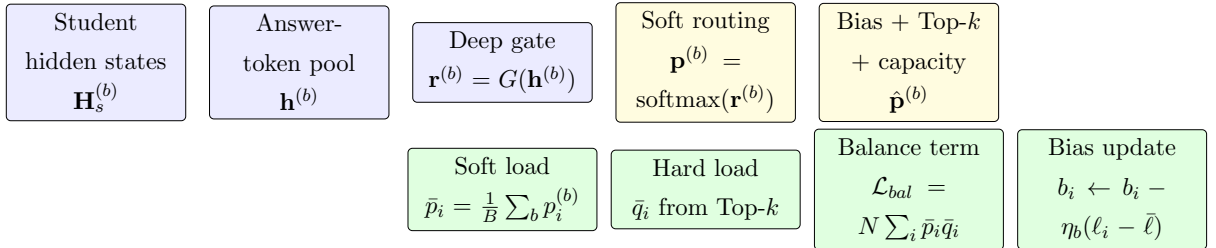


Figure 3.3: Student-conditioned routing in GRACE. Pooled student features are transformed into gate logits, converted into routing weights, and sparsified by top- k and capacity constraints. The full framework can additionally apply a balance term and expert-bias updates.

Figure 3.5 gives a routing example inspired by the routing-overflow and load-balancing behavior used in sparse expert models [46, 47]. The dense gate may prefer a small subset of teachers, but top- k selection, capacity constraints, and fallback convert these scores into a sparse assignment pattern with explicit load statistics.

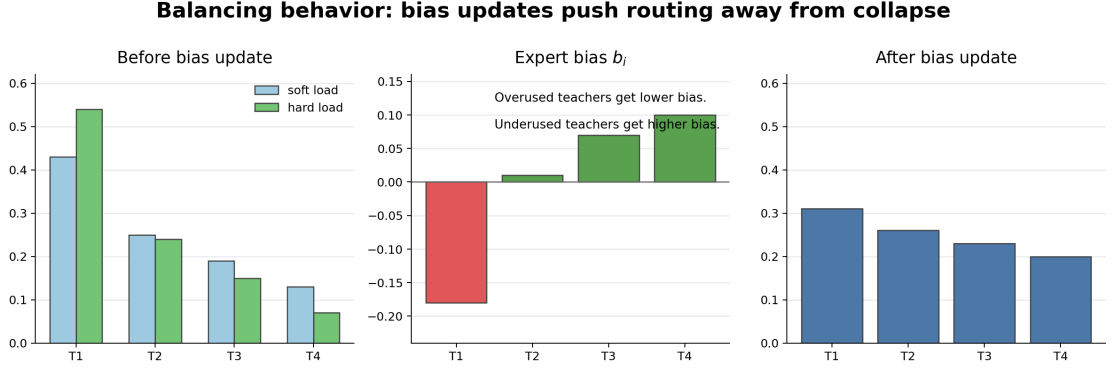


Figure 3.4: Bias-feedback behavior in GRACE when expert-bias updates are enabled. If one teacher receives too many assignments, its bias is reduced, while underused teachers receive higher bias. This shifts future routing toward a more balanced load.

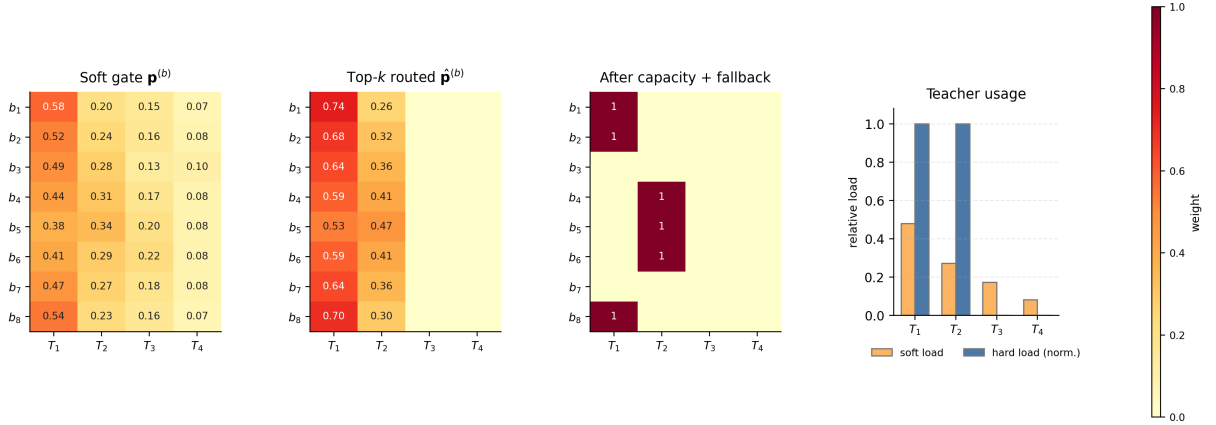


Figure 3.5: Routing dynamics in GRACE. Left to right: dense gate scores, top- k routed weights, and the final assignment pattern after capacity control and fallback. The rightmost panel compares average soft load and normalized hard load, illustrating why balancing terms and expert-bias updates are needed.

3.4 Gradient Alignment Filtering

3.4.1 Gradient Proxies

For each sample b , let $\mathcal{L}_{CE}^{(b)}$ be the student cross-entropy loss on supervised answer tokens. We define the CE logit-gradient proxy as

$$\mathbf{g}_{CE}^{(b)} = \nabla_{\mathbf{z}_s^{(b)}} \mathcal{L}_{CE}^{(b)}. \quad (3.16)$$

In implementation, this gradient is obtained in closed form from the student probability distribution and target labels, which avoids an additional backward pass through the full model.

For each teacher T_i , the KD logit-gradient proxy is

$$\mathbf{g}_i^{(b)} = \nabla_{\mathbf{z}_s^{(b)}} \mathcal{L}_{KD}^{(b,i)}. \quad (3.17)$$

This proxy is computed directly from the selected distillation loss. In the default setting of this thesis, $\mathcal{L}_{KD}^{(b,i)}$ is the ULD loss in Equation 3.6.

3.4.2 Alignment Score

For each sample b and teacher T_i , we measure directional agreement between supervised learning and KD by cosine similarity:

$$\tilde{\mathbf{g}}_{CE}^{(b)} = \text{Pool}\left(\mathbf{g}_{CE}^{(b)}\right), \quad \tilde{\mathbf{g}}_i^{(b)} = \text{Pool}\left(\mathbf{g}_i^{(b)}\right), \quad (3.18)$$

$$s_i^{(b)} = \cos\left(\tilde{\mathbf{g}}_{CE}^{(b)}, \tilde{\mathbf{g}}_i^{(b)}\right). \quad (3.19)$$

Here, $\text{Pool}(\cdot)$ averages the logit-gradient tensor over supervised answer positions, producing one pooled CE gradient and one pooled KD gradient per teacher. This is the quantity used by the current implementation.

3.4.3 Warmup, Thresholding, and Fallback

The alignment filter is enabled only after an optional warmup phase. Let $\rho \in [0, 1]$ denote the warmup ratio with respect to the total number of training steps. During the first ρ fraction of training, the method uses only the constrained gate weights $\hat{p}_i^{(b)}$. After warmup, a teacher is considered alignment-active when

$$a_i^{(b)} = \mathbf{1}[s_i^{(b)} > \delta], \quad (3.20)$$

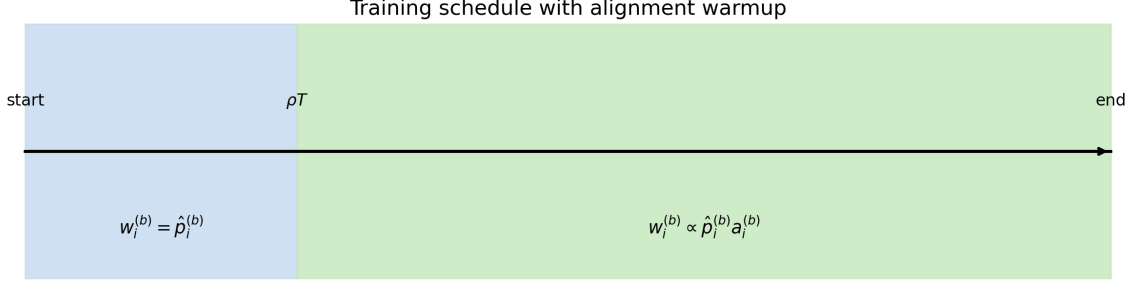


Figure 3.6: Warmup schedule used by GRACE. During the early stage, the method trusts the routed weights $\hat{p}_i^{(b)}$ directly. After the warmup boundary ρT , the alignment filter is activated and the final effective weights depend on both routing and alignment.

where δ is the gradient-alignment threshold.

After warmup and thresholding have identified the alignment-active teachers, the alignment filter is applied on top of the already routed teachers. If at least one routed teacher satisfies Equation 3.20, the final effective teacher weights are

$$w_i^{(b)} = \frac{\hat{p}_i^{(b)} a_i^{(b)}}{\sum_{j=1}^N \hat{p}_j^{(b)} a_j^{(b)}}. \quad (3.21)$$

If thresholding removes all routed teachers, GRACE does not drop KD for that sample. Instead, it falls back to the routed teacher with the largest alignment score:

$$i^* = \arg \max_{i: \hat{p}_i^{(b)} > 0} s_i^{(b)}, \quad a_i^{(b)} = \mathbf{1}[i = i^*]. \quad (3.22)$$

Equation 3.21 is then recomputed with this fallback mask. As a result, the current implementation never drops KD entirely for a sample after routing; instead, it keeps the least-negative routed teacher when necessary.

Figure 3.8 shows the same rule from a weight-update view. The routed gate weights are first proposed by the student-conditioned router, then refined by alignment scores, and finally converted into the effective KD weights used in Equation 3.23.

3.4.4 Sample-Wise KD Aggregation

The routed KD loss for sample b is the weighted aggregation

$$\bar{\mathcal{L}}_{KD}^{(b)} = \sum_{i=1}^N w_i^{(b)} \mathcal{L}_{KD}^{(b,i)}. \quad (3.23)$$

The batch-level KD loss is then the mean over samples:

$$\mathcal{L}_{KD} = \frac{1}{B} \sum_{b=1}^B \bar{\mathcal{L}}_{KD}^{(b)}. \quad (3.24)$$

Gradient-alignment behavior in GRACE

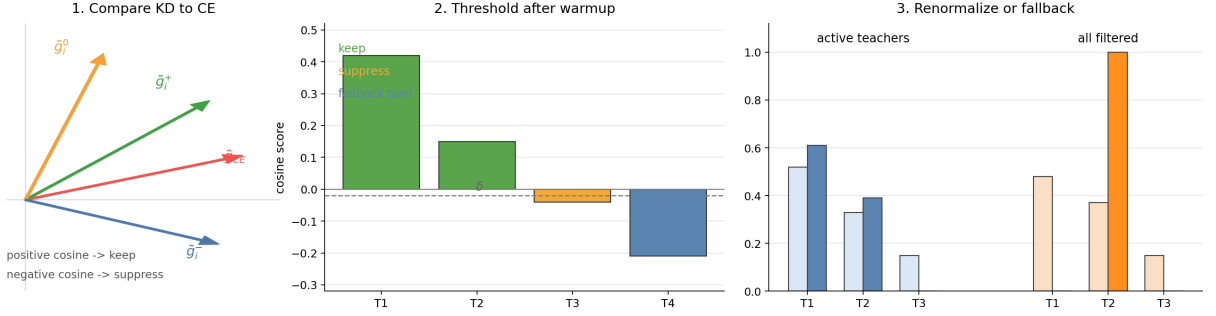


Figure 3.7: Intuition for gradient alignment in GRACE. Left: the CE gradient provides the reference update direction, and each teacher is judged by the cosine similarity of its pooled KD gradient with that reference. Middle: after warmup, teachers above the threshold δ remain active, while weakly or negatively aligned teachers are suppressed. Right: if at least one teacher survives, the routed weights are renormalized over the active set; otherwise the least-negative routed teacher becomes the fallback expert.

Alignment refines routed teachers into effective KD weights

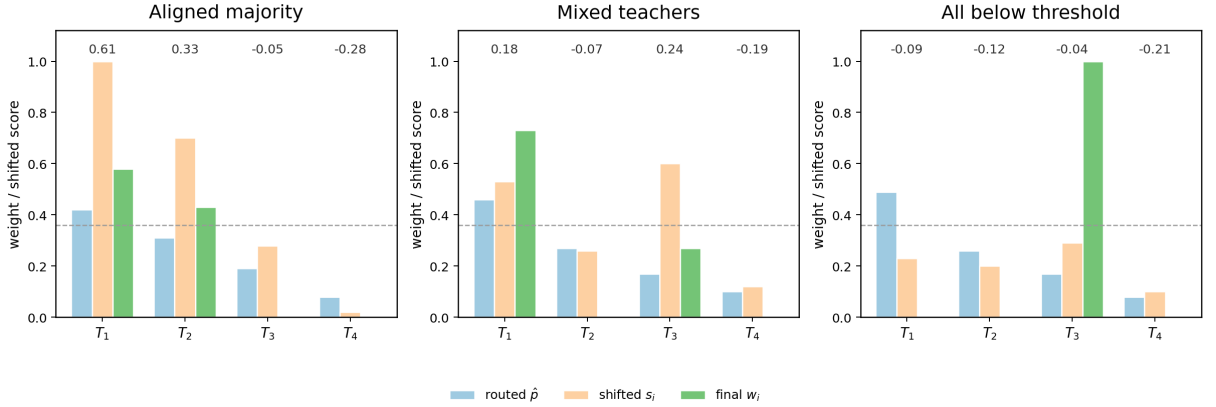


Figure 3.8: Three alignment cases in GRACE. When most routed teachers agree with the CE direction, KD remains distributed across them. Under mixed agreement, the final weights concentrate on the positively aligned teachers. When all routed teachers fall below the threshold, the least-negative routed teacher becomes the fallback expert.

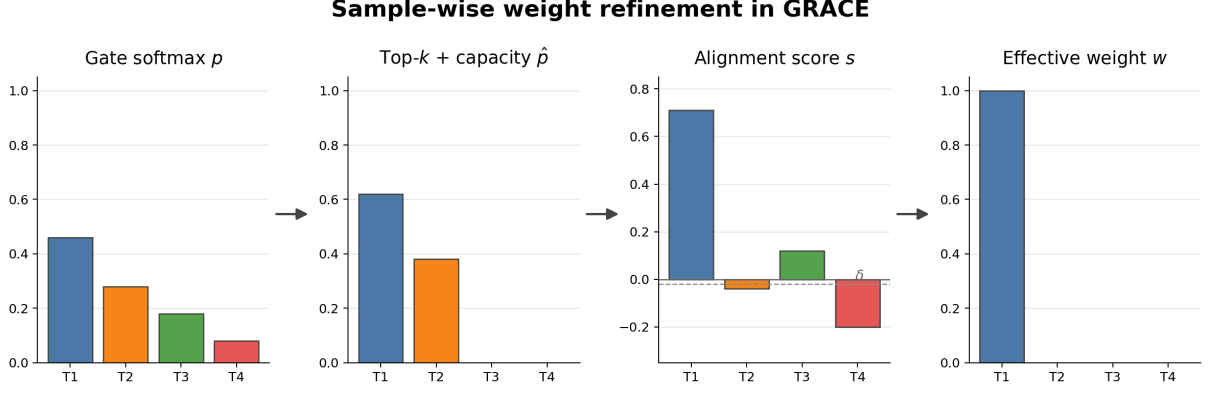


Figure 3.9: A sample-wise view of weight refinement in GRACE. The gate first proposes dense soft weights, top- k and capacity make them sparse, alignment scores test whether routed teachers are useful, and the remaining mass becomes the final KD weights.

This rule has two stages: the gate proposes sparse teacher weights, and gradient alignment filters the routed teachers before the final weighted KD average is formed.

3.5 Final Objective

The final optimization objective is

$$\mathcal{L} = (1 - \alpha)\mathcal{L}_{CE} + \alpha\mathcal{L}_{KD} + \lambda_{bal}\mathcal{L}_{bal}, \quad (3.25)$$

where $\mathcal{L}_{CE}$ is the standard supervised next-token cross-entropy loss and $\mathcal{L}_{KD}$ is defined in Equation 3.24. The balance term $\mathcal{L}_{bal}$ is defined in Equation 3.14 and is used only when $\lambda_{bal} > 0$.

Equation 3.25 keeps the student anchored to ground-truth supervision while adding routed KD and, when enabled, an auxiliary balancing term. Compared with static multi-teacher averaging, this objective makes the teacher contribution adaptive at the sample level. Compared with fixed teacher-subset selection, it avoids committing to one subset for the whole run [43, 44, 48].

Chapter 4

Evaluation Methodology

4.1 Experimental Setup

All experiments follow a shared evaluation protocol for fair comparison across methods.

The study focuses on two compact student models, **SmolVLM-500M** and **SmolVLM-256M**, evaluated on text-rich multimodal benchmarks [31].

All model evaluations are conducted using **VLM-Eval-Kit** [53]. In brief, the toolkit works as follows:

1. Load a benchmark split and apply its prompt template or input format.
2. Run model inference through a unified model wrapper interface.
3. Post-process raw generations into normalized answers.
4. Compute dataset-specific scores (e.g., VQA Accuracy, ANLS, Relaxed Accuracy) and aggregate final results.

This unified pipeline ensures a consistent evaluation procedure across all compared models.

4.2 Datasets

We evaluate three text-centric VQA benchmarks: **TextVQA**, **DocVQA**, and **ChartQA**.

The goal is to measure whether the model can **read textual content in images, understand questions, and perform basic reasoning** across different visual domains [7–9].

These datasets are selected because they require OCR-heavy understanding, cover complementary visual domains, and include retrieval, layout understanding, and numerical reasoning [7–9].

4.2.1 Dataset Descriptions

TextVQA [7]: evaluates question answering on natural images containing scene text. It focuses on reading text in-the-wild and combining it with question understanding. *Sample question*: “What word is written on the stop sign?” *Sample answer*: “No word is visible on this sign.”



Figure 4.1: Example image for a TextVQA-style question [15].

DocVQA [8]: evaluates VQA on real document images (scans/photos), where layout, paragraphs, tables and forms are important. It emphasizes document structure understanding in addition to OCR. *Sample question*: “What is the invoice number shown in this document?” *Sample answer*: “4572.”

ChartQA [9]: evaluates question answering over charts. It tests chart reading plus numerical and logical reasoning to derive the final answer. *Sample question*: “Which year has the highest sales value in the chart?” *Sample answer*: “The second (middle) bar has the highest value.”

4.2.2 Dataset Size and Splits

4.3 Training Configuration

Unless otherwise stated, the main distillation experiments use a shared training configuration so that differences in performance can be attributed to the distillation strategy rather than unrelated optimization choices.

All training runs are executed on a single **NVIDIA A100 80GB** GPU using the **Modal** runtime environment. Distillation training uses **DeepSpeed ZeRO Stage 2**, **bf16** mixed



Figure 4.2: Example document image for a DocVQA-style question [16].

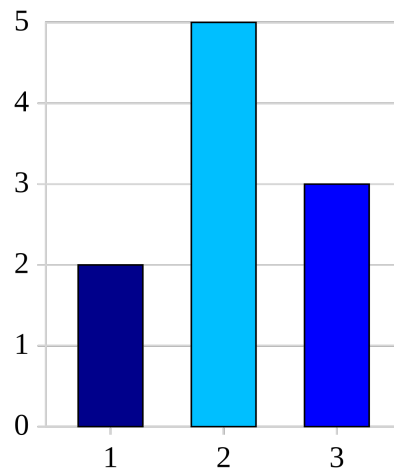


Figure 4.3: Example chart image for a ChartQA-style question [17].

Dataset	Train	Val	Test	Total
TextVQA	34,602	5,000	5,734	45,336
DocVQA	39,463	5,349	5,188	50,000
ChartQA-H (human)	7,398	960	1,250	9,608
ChartQA-M (machine)	20,901	960	1,250	23,111
ChartQA (H+M, combined)	28,299	1,920	2,500	32,719

Table 4.1: Dataset split sizes (number of QA pairs) for TextVQA, DocVQA and ChartQA [7–9].

precision, **TF32** matrix multiplication, and gradient checkpointing to control memory use while keeping the student fully trainable and all teachers frozen [54, 55].

Before training, all benchmark samples are converted into a unified **LLaVA-style multimodal instruction format** [25]. Each example is stored as an image plus a short user-assistant dialogue, so TextVQA, DocVQA, and ChartQA can share the same data loader, prompt template, and answer supervision interface.

Listing 4.1 shows the structure used in our preprocessing pipeline.

```

1 {
2   "image": "sample_000123.jpg",
3   "conversations": [
4     {"from": "human", "value": "<image>\nWhat is the invoice number?"},
5     {"from": "gpt", "value": "4572"}
6   ]
7 }
```

Listing 4.1: Representative LLaVA-style sample used after dataset conversion.

4.3.1 Offline Teacher-Logit Preparation

Teacher logits are generated *offline* before distillation. An offline caching script runs each frozen teacher on every training example, keeps only the supervised answer-token positions, and writes one `.pt` file per sample under a teacher-specific directory. Each record stores the sample index, the selected teacher logits, and the corresponding supervised labels. This keeps the online training loop simple: the student only reads precomputed teacher targets instead of running the teachers again.

After generation, the teacher-logit directories are uploaded to a **Cloudflare R2** bucket using its S3-compatible object-storage interface [56, 57]. In our setup, the bucket already exceeds **1 TB**, so copying the whole cache to a rented GPU instance is impractical. This

is especially relevant on services such as Vast.ai, where local NVMe space is usually much smaller than the full teacher logit store.

4.3.2 Remote Logit Streaming During Training

The current main training path does *not* predownload the full cache. When an R2 root is provided to the launcher `distill_multi_teachers.sh`, the trainer switches to remote lazy loading. During training, `StreamingTeacherLogitsCache` resolves the teacher-specific remote path

`remote_root/<dataset>_teacher_logits_<alias>/<teacher-slug>/<index>.pt,`

then fetches files lazily from R2 through `s3fs` and `WholeFileCacheFileSystem` [58, 59]. The repository also includes a repacking utility for MDS, but the current main training path reads the raw per-sample `.pt` cache directly.

To keep disk usage bounded, the loader uses a **100 GB** local cache. Each file is cached only after first access. After a file is opened, the loader updates its access time. When the total cache size exceeds the limit, the oldest cached files are removed first. In effect, the local cache behaves as a small least-recently-used style buffer over the much larger R2 store. A per-teacher file lock is also used when cache metadata is updated, so concurrent dataloader workers do not corrupt the cache state.

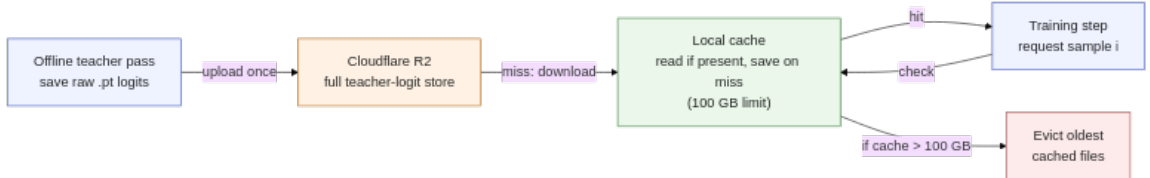


Figure 4.4: Training with remote teacher logits. Offline caches are uploaded to R2, fetched lazily during training, and kept in a bounded local cache with oldest-file eviction.

4.4 Evaluation Metrics

We report benchmark-specific official metrics for each dataset:

- **TextVQA: VQA Accuracy.** For a predicted answer $\hat{a}$, the per-question score is

$$s_{\text{VQA}} = \min \left(\frac{\#\text{annotators that provided } \hat{a}}{3}, 1 \right),$$

and the final score is the mean over all questions [7].

Setting	Value
Student model	HuggingFaceTB/SmolVLM-500M-Instruct
Teacher models	Qwen/Qwen2.5-VL-3B-Instruct google/gemma-3-4b-it
Teacher weighting strategy	routing
Distillation loss	ULD
Training dataset	DocVQA
Validation split	validation
Hardware and runtime	1× NVIDIA A100 80GB GPU on Modal
Distributed training	DeepSpeed ZeRO-2
Optimizer	adamw_torch
Adam betas	(0.9, 0.98)
Adam epsilon	1×10^{-7}
Weight decay	N/A
Numerical precision	bf16 enabled, fp16 disabled, TF32 enabled
FlashAttention-2	Enabled
Training data format	LLaVA-style multimodal instruction JSON
Teacher-logit storage	Cloudflare R2 object storage
Teacher-logit format	Raw per-sample .pt files
Local cache directory	/tmp/teacher-logits-streaming
Local cache limit	100 GB
Max sequence length	16384
Training epochs	1
Per-device batch size	20
Gradient accumulation steps	1
Effective batch size	20
Base learning rate	1×10^{-5}
Vision learning rate	2×10^{-6}
Connector learning rate	1×10^{-5}
Learning-rate schedule	Cosine decay with warmup ratio 0.03
Gradient checkpointing	Enabled
Frozen modules	None
Dataloader workers	4
Logging and checkpointing	Log every 5 steps; save every 150 steps
Student/teacher temperature	$\tau_s = \tau_t = 1.0$
KD weight	$\alpha = 0.5$
Top- k routing	$k = 2$
Capacity factor	$\phi = 1.25$
Balance weight	$\lambda_{bal} = 0.01$
Bias-update rate	$\eta_b = 5 \times 10^{-4}$
Alignment warmup ratio	$\rho = 0.2$
Alignment threshold	$\delta = -0.02$
Seed	N/A
Image resolution / visual token budget	N/A

Table 4.2: Training configuration for the current main two-teacher distillation script. The table mixes fixed launch-script values with a small number of placeholders that can be finalized later.

- **DocVQA: ANLS** (Average Normalized Levenshtein Similarity):

$$\text{NLS}(a, \hat{a}) = 1 - \frac{d_{\text{Lev}}(a, \hat{a})}{\max(|a|, |\hat{a}|)}, \quad s_{\text{ANLS}} = \begin{cases} \text{NLS}(a, \hat{a}), & \text{NLS} \geq 0.5, \\ 0, & \text{otherwise.} \end{cases}$$

The final ANLS is the average score over all samples [8].

- **ChartQA: Relaxed Accuracy.** A prediction is correct if it matches exactly for text answers, or if the relative error is within 5% for numeric answers:

$$\frac{|\hat{y} - y|}{|y|} \leq 0.05.$$

The final score is the proportion of correct predictions [9].

4.5 Baselines and Compared Methods

For consistency with Chapter 5, we organize the compared methods into four groups: non-distillation references, same-family distillation baselines, different-family single-teacher baselines, and heterogeneous multi-teacher baselines.

Group	Methods	Teacher setup	Purpose
Non-distillation references	Original, full finetuning	No teacher	Student-only reference points
Same-family baselines	Forward KL, Reverse KL, Jensen-Shannon, HSM-CKA	SmolVLM2-2.2B-Instruct	Standard single-teacher objectives within the SmolVLM family
Different-family single-teacher baselines	ULD, layer-wise hidden-state loss	Qwen2-VL-2B-Instruct or Gemma-4B	Cross-family transfer without multi-teacher routing
Heterogeneous multi-teacher baselines	Uniform averaging, reinforced selection, GRACE	Mixed teacher pool from multiple VLM families	Static averaging vs. stochastic policy selection vs. adaptive teacher assignment

Table 4.3: Baseline groups used in the evaluation chapter.

- **Non-distillation references.** The original student and the full-finetuning baseline provide the student-only reference points for each benchmark.
- **Same-family baselines.** Forward KL, reverse KL, Jensen-Shannon divergence, and HSM-CKA all use **SmolVLM2-2.2B-Instruct** as a same-family teacher [11, 31, 37, 49, 60, 61]. They isolate standard teacher-student transfer when tokenizer, vocabulary, and output space are already matched.

- **Different-family single-teacher baselines.** This group uses one teacher outside the SmolVLM family. ULD is the main cross-tokenizer logits baseline [14], while the layer-wise hidden-state baseline tests whether direct internal matching remains useful across heterogeneous VLM families [37].
- **Heterogeneous multi-teacher baselines.** Uniform averaging, reinforced selection, and GRACE all use teacher pools drawn from different VLM families. Uniform averaging keeps fixed equal weights, reinforced selection learns a stochastic teacher policy with REINFORCE [43, 62], and GRACE uses sparse routing plus alignment-aware reweighting [44, 45].

For the trainable competitor methods, we use a shared supervised term $\mathcal{L}_{CE}$ and vary only the distillation component. A convenient generic form is

$$\mathcal{L}_{\text{base}} = (1 - \alpha)\mathcal{L}_{CE} + \alpha\mathcal{L}_{KD}. \quad (4.1)$$

The compared objectives are:

- **Original:** direct evaluation of the released student, with no additional optimization. This measures the native capability of the compact model before any task adaptation or distillation.
- **Full finetuning:** $\mathcal{L} = \mathcal{L}_{CE}$. This baseline uses only ground-truth supervision and no teacher, so it isolates the benefit of external teacher knowledge.
- **Forward KL:** this is the standard logits-based KD baseline, where the student matches the teacher soft distribution over the shared vocabulary [11]. It is appropriate for same-family transfer because both models use compatible token spaces:

$$\mathcal{L}_{KD} = \text{KL}(p_t \| p_s).$$

- **Reverse KL:** this baseline swaps the divergence direction and penalizes student probability mass that falls outside the teacher support. In KD studies, it is often treated as a sharper, more mode-seeking alternative to forward KL [61, 63]:

$$\mathcal{L}_{KD} = \text{KL}(p_s \| p_t).$$

- **Jensen-Shannon:** this baseline symmetrizes forward and reverse KL by comparing both distributions to their midpoint. It is a standard compromise when one wants a bounded and symmetric divergence on the same token space [49, 61]:

$$\mathcal{L}_{KD} = \frac{1}{2}\text{KL}(p_s \| m) + \frac{1}{2}\text{KL}(p_t \| m), \quad m = \frac{1}{2}(p_s + p_t).$$

- **HSM-CKA.**

This baseline adds linear-CKA hidden-state matching to same-family logits KD [37, 64]:

$$\mathcal{L} = (1 - \alpha)\mathcal{L}_{CE} + \alpha\mathcal{L}_{\text{Fwd-KL}} + \lambda_H \left(1 - \sqrt{\text{CKA}(H_s, H_t)}\right).$$

- **ULD:** this is the main cross-family logits baseline. Rather than aligning token IDs directly, it sorts the probability mass and compares ranked distributions, which makes it suitable when the teacher and student have different vocabularies or tokenizers [14]:

$$\mathcal{L}_{KD} = \|\pi(p_s) - \pi(p_t)\|_1.$$

- **Layer-wise hidden-state loss:** this baseline extends hidden-state matching across multiple intermediate layers and tests whether explicit internal alignment is sufficient in the heterogeneous setting. It keeps the logits objective but adds representation terms over selected layers [37]:

$$\mathcal{L} = (1 - \alpha)\mathcal{L}_{CE} + \alpha\mathcal{L}_{KD} + \lambda_H \sum_{\ell} \left(1 - \sqrt{\text{CKA}(H_s^{(\ell)}, H_t^{(\ell)})}\right).$$

- **Uniform multi-teacher:** this baseline uses the same heterogeneous teacher pool as GRACE, but removes all adaptivity. Each teacher contributes equally, so it tests whether teacher diversity alone is sufficient without routing or selection [43, 44]:

$$\mathcal{L}_{KD} = \frac{1}{M} \sum_{i=1}^M \mathcal{L}_{KD}^{(i)}.$$

- **Reinforced selection:** this baseline keeps the multi-teacher setting but replaces deterministic routing with a stochastic teacher-selection policy. The selector predicts Bernoulli probabilities, samples a subset, normalizes the selected teachers, and is optimized with a REINFORCE-style reward signal [43, 62]:

$$\pi_i = \sigma(u_i), \quad m_i \sim \text{Bernoulli}(\pi_i), \quad w_i = \frac{m_i}{\sum_j m_j},$$

then uses

$$\mathcal{L}_{KD} = \sum_{i=1}^M w_i \mathcal{L}_{KD}^{(i)}, \quad \mathcal{L} = (1 - \alpha)\mathcal{L}_{CE} + \alpha\mathcal{L}_{KD} + \lambda_{\text{pol}}\mathcal{L}_{\text{pol}},$$

with a policy term $\mathcal{L}_{\text{pol}} = -A \log p(m \mid \pi)$ and a fallback to the highest-probability teacher when $\sum_i m_i = 0$.

Unless noted otherwise, all baselines reuse the same data conversion pipeline, hardware, optimizer family, and checkpointing protocol described earlier in this chapter. Methods that do not use routing simply omit the GRACE-specific terms and retain the corresponding single-teacher or uniform multi-teacher objective from the cited source.

Chapter 5

Experiment Results

5.1 Main Comparison Results

This section compares the proposed method with the baselines on TextVQA, DocVQA, and ChartQA.

5.1.1 Baseline Results

Before applying adaptation or distillation, we report the original performance of the two student models, **SmolVLM-500M** and **SmolVLM-256M**.

These baseline results establish the starting point for later comparison.

All results are obtained using **VLM-Eval-Kit** [53] under the same evaluation pipeline described in Chapter 4.

Model	TextVQA	DocVQA	ChartQA
SmolVLM-500M	56.75	59.5247	62.8
SmolVLM-256M	47.17	48.6411	45.04

Table 5.1: Original benchmark results of the two SmolVLM models evaluated with VLM-Eval-Kit.

Teacher Model	TextVQA	DocVQA	ChartQA
Qwen2-VL-2B-Instruct	79.7	90.1	73.5
Qwen2.5-VL-3B-Instruct	79.3	93.9	81.2
InternVL3-1B	74.1	81.9	75.3
Gemma-3-4B-it	57.8	75.8	63.6

Table 5.2: Original benchmark results of the teacher models used in the heterogeneous teacher pool, compiled from the official model releases [18, 19] and [20, 21] and, for the Qwen2.5-VL-3B ChartQA entry, from a model-comparison page that reports the base-model metric [22].

5.1.2 Overall Comparison

Tables 5.3, 5.4, and 5.5 summarize the main comparison results across original models, direct fine-tuning, standard logits distillation baselines, and GRACE.

Missing entries are marked as ‘N/A’ when a run was not completed or not reported in the current experiment set.

We report the baselines in four groups.

- The **non-distillation references** are the original student and the full-finetuning baseline.
- The **same-family baselines** use **SmolVLM2-2.2B-Instruct** as teacher for forward KL, reverse KL, Jensen-Shannon divergence, and HSM-CKA [11, 36, 37].
- The **different-family single-teacher baselines** use a teacher outside the SmolVLM family. This group includes the ULD baseline [14] and the layer-wise hidden-state baseline.
- The **heterogeneous multi-teacher baselines** use teacher pools drawn from different VLM families, including uniform averaging, reinforced selection, and GRACE [43–45, 62].

5.2 Representation Similarity Analysis

We use CKA as an analysis tool to examine whether teacher and student layers exhibit clear correspondence. For centered hidden-state matrices $\tilde{\mathbf{H}}_s$ and $\tilde{\mathbf{H}}_t$, linear CKA is

$$\text{CKA}(\tilde{\mathbf{H}}_s, \tilde{\mathbf{H}}_t) = \frac{\left\| \tilde{\mathbf{H}}_t^\top \tilde{\mathbf{H}}_s \right\|_F^2}{\left\| \tilde{\mathbf{H}}_t^\top \tilde{\mathbf{H}}_t \right\|_F \left\| \tilde{\mathbf{H}}_s^\top \tilde{\mathbf{H}}_s \right\|_F}. \quad (5.1)$$

Values closer to 1 indicate stronger representation similarity [64].

The main question is whether strong internal similarity is enough to justify layer-wise distillation. Our results suggest that it is not, as shown in the figures and tables below.

Figures 5.1 and 5.2 show that the teacher and student can be relatively close in both the language and vision stacks.

However, Tables 5.3, 5.4, and 5.5 still show a clear benchmark gap between compact students and stronger models. The layer-wise baseline also fails to consistently outperform output-level distillation.

Category	Method	Teacher	Student	TextVQA
Non-distillation references				
Original	None	None	SmolVLM-500M	56.75
			SmolVLM-256M	47.17
Reference	Finetuning	None	SmolVLM-500M	56.96
			SmolVLM-256M	49.832
Same-family distillation baselines				
Same-family	Forward KL	SmolVLM2-2.2B	SmolVLM-500M	59.0
			SmolVLM-256M	50.88
	Reverse KL	SmolVLM2-2.2B	SmolVLM-500M	44.734
			SmolVLM-256M	29.752
	Jensen-Shannon	SmolVLM2-2.2B	SmolVLM-500M	N/A
			SmolVLM-256M	N/A
HSM-CKA	SmolVLM2-2.2B	SmolVLM-500M	N/A	
		SmolVLM-256M	N/A	
Different-family single-teacher baselines				
Cross-family	ULD	Qwen2-VL-2B-Instruct	SmolVLM-500M	61.49
			SmolVLM-256M	53.946
	Layer-wise loss	Qwen2-VL-2B-Instruct	SmolVLM-500M	N/A
			SmolVLM-256M	N/A
Heterogeneous multi-teacher baselines				
Multi-teacher	Uniform	Qwen2-VL-2B-Instruct	SmolVLM-500M	63.88
		Qwen2.5-VL-3B-Instruct	SmolVLM-256M	54.57
	Reinforced selection	Qwen2-VL-2B-Instruct		
		Qwen2.5-VL-3B-Instruct	SmolVLM-500M	N/A
		InternVL3-1B	SmolVLM-256M	N/A
		Gemma-3-4B-it		
	GRACE	Qwen2-VL-2B-Instruct		
		Qwen2.5-VL-3B-Instruct	SmolVLM-500M	N/A
		InternVL3-1B	SmolVLM-256M	N/A
Gemma-3-4B-it				

Table 5.3: TextVQA comparison across original models, fine-tuning, distillation baselines, and the proposed method.

Category	Method	Teacher	Student	DocVQA
Non-distillation references				
Original	None	None	SmolVLM-500M	59.5247
			SmolVLM-256M	48.6411
Reference	Finetuning	None	SmolVLM-500M	61.9634
			SmolVLM-256M	42.0272
Same-family distillation baselines				
Same-family	Forward KL	SmolVLM2-2.2B	SmolVLM-500M	63.5517
			SmolVLM-256M	49.46
	Reverse KL	SmolVLM2-2.2B	SmolVLM-500M	29.28
			SmolVLM-256M	23.2811
	Jensen-Shannon	SmolVLM2-2.2B	SmolVLM-500M	N/A
			SmolVLM-256M	N/A
HSM-CKA	SmolVLM2-2.2B	SmolVLM-500M	N/A	
		SmolVLM-256M	N/A	
Different-family single-teacher baselines				
Cross-family	ULD	Qwen2-VL-2B-Instruct	SmolVLM-500M	65.2832
			SmolVLM-256M	52.2424
	Layer-wise loss	Qwen2-VL-2B-Instruct Gemma-4B	SmolVLM-500M	63.3218 (gradnorm)
			SmolVLM-256M	N/A
			SmolVLM-500M	N/A
Heterogeneous multi-teacher baselines				
Multi-teacher	Uniform	Qwen2-VL-2B-Instruct	SmolVLM-500M	69.59
		Qwen2.5-VL-3B-Instruct	SmolVLM-256M	55.7036
	Reinforced selection	Qwen2-VL-2B-Instruct		
		Qwen2.5-VL-3B-Instruct	SmolVLM-500M	N/A
		InternVL3-1B	SmolVLM-256M	N/A
		Gemma-3-4B-it		
	GRACE	Qwen2-VL-2B-Instruct		
		Qwen2.5-VL-3B-Instruct	SmolVLM-500M	N/A
		InternVL3-1B	SmolVLM-256M	N/A
		Gemma-3-4B-it		

Table 5.4: DocVQA comparison across original models, fine-tuning, distillation baselines, and the proposed method.

Category	Method	Teacher	Student	ChartQA
Non-distillation references				
Original	None	None	SmolVLM-500M	62.8
			SmolVLM-256M	45.04
Reference	Finetuning	None	SmolVLM-500M	63.0
			SmolVLM-256M	49.68
Same-family distillation baselines				
Same-family	Forward KL	SmolVLM2-2.2B	SmolVLM-500M	64.52
			SmolVLM-256M	56.8
	Reverse KL	SmolVLM2-2.2B	SmolVLM-500M	30.16
			SmolVLM-256M	17.2
	Jensen-Shannon	SmolVLM2-2.2B	SmolVLM-500M	N/A
			SmolVLM-256M	N/A
HSM-CKA	SmolVLM2-2.2B	SmolVLM-500M	N/A	
		SmolVLM-256M	N/A	
Different-family single-teacher baselines				
Cross-family	ULD	Qwen2-VL-2B-Instruct	SmolVLM-500M	66.92
			SmolVLM-256M	60.96
	Layer-wise loss	Qwen2-VL-2B-Instruct Gemma-4B	SmolVLM-500M	76.88
			SmolVLM-256M	62.4
			SmolVLM-500M	76.08
Heterogeneous multi-teacher baselines				
Multi-teacher	Uniform	Qwen2-VL-2B-Instruct	SmolVLM-500M	75.6
		Qwen2.5-VL-3B-Instruct	SmolVLM-256M	63.68
	Reinforced selection	Qwen2-VL-2B-Instruct		
		Qwen2.5-VL-3B-Instruct	SmolVLM-500M	N/A
		InternVL3-1B	SmolVLM-256M	N/A
		Gemma-3-4B-it		
	GRACE	Qwen2-VL-2B-Instruct		
		Qwen2.5-VL-3B-Instruct	SmolVLM-500M	79.2
InternVL3-1B		SmolVLM-256M	N/A	
		Gemma-3-4B-it		

Table 5.5: ChartQA comparison across original models, fine-tuning, distillation baselines, and the proposed method.

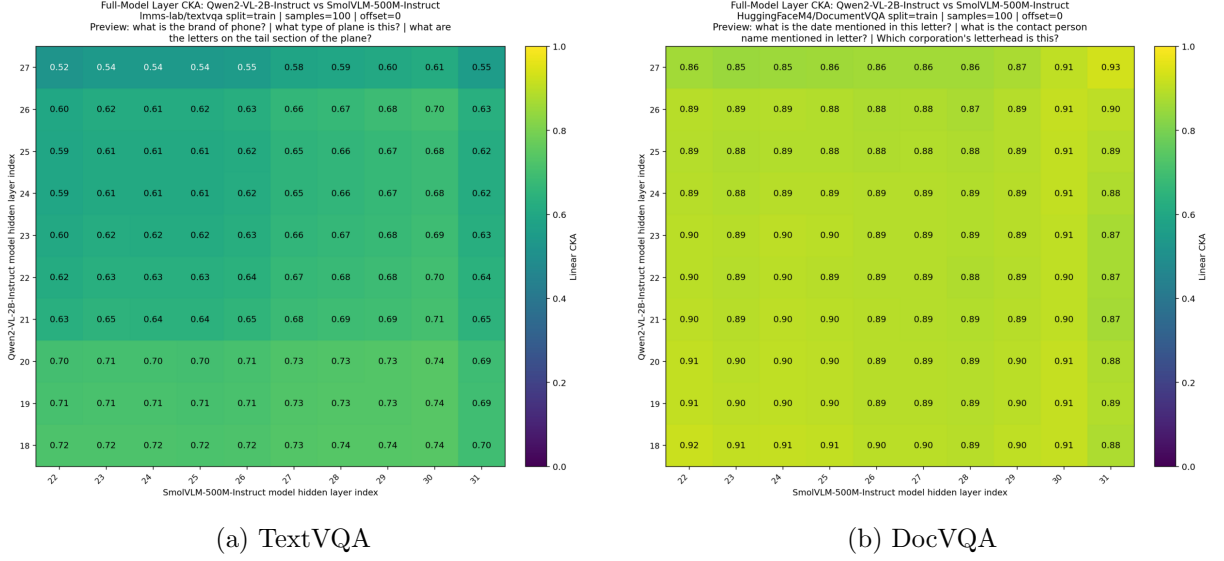


Figure 5.1: Final-layer CKA heatmaps from our experiments for the teacher-student pair Qwen2-VL-2B and SmolVLM-500M on TextVQA and DocVQA. Brighter regions and values closer to 1 indicate stronger representation similarity.

This supports the design choice in GRACE. High hidden-state similarity alone is not a reliable signal that direct layer matching will produce better task transfer.

5.3 Ablation Study

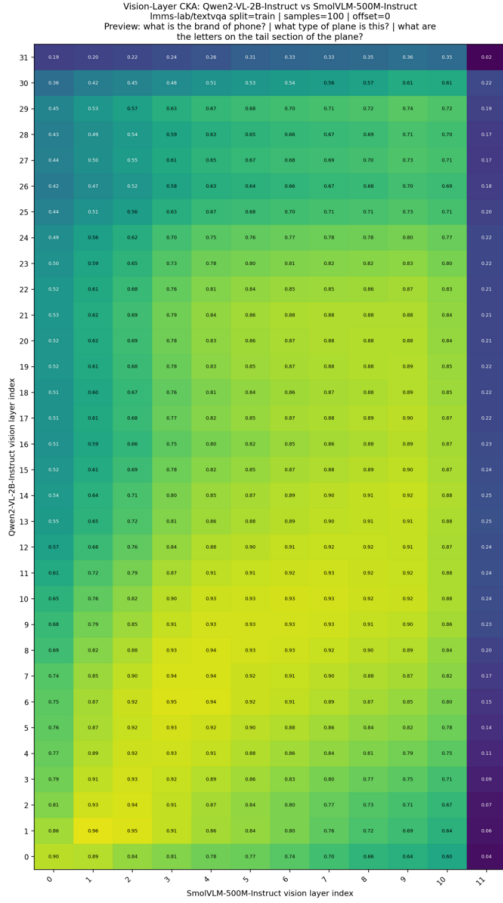
This section analyzes the effect of teacher assignment, balancing, and gradient alignment on final performance.

5.4 Qualitative Analysis

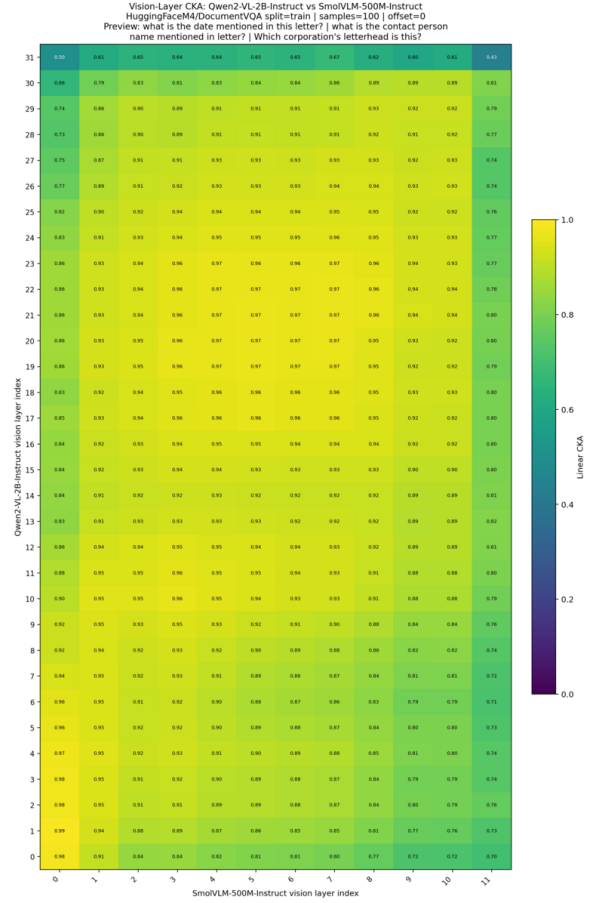
This section presents representative examples to inspect how distillation affects text reading and basic reasoning behavior.

5.5 Discussion

This section discusses the trade-off between model size, teacher supervision, and transfer performance across the evaluated benchmarks.

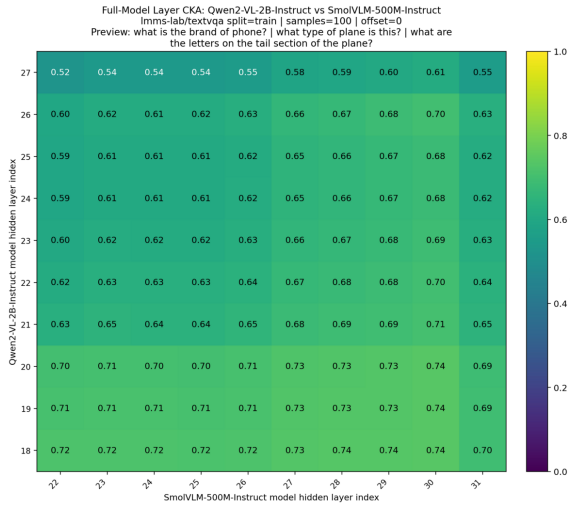


(a) TextVQA

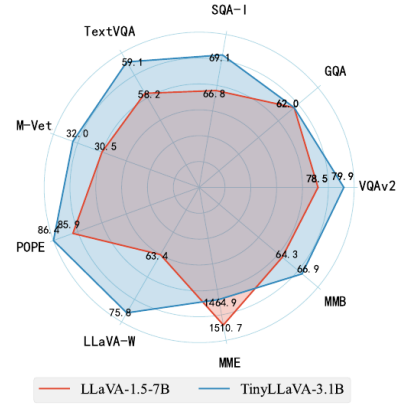


(b) DocVQA

Figure 5.2: Vision-layer CKA heatmaps for the same teacher-student pair on TextVQA and DocVQA. Again, brighter regions and values closer to 1 indicate stronger similarity.



(a) High internal similarity in CKA space.



(b) A persistent benchmark gap remains for compact VLMs [28].

Figure 5.3: High CKA does not by itself imply that layer-wise transfer will close the downstream performance gap. Representation similarity and task transfer are related, but not equivalent.

Conclusion and Future Work

Chapter A

PyTorch-Style GRACE Reference

This appendix keeps only the core GRACE logic. The code is written as PyTorch-style pseudocode, but the flow follows the actual repository implementation.

A.1 Core Gate

```
1 class GRACEGate(nn.Module):
2     def __init__(self, hidden_size, num_teachers):
3         super().__init__()
4         self.router = MLP(hidden_size, num_teachers)
5         self.expert_bias = nn.Parameter(torch.zeros(num_teachers),
6                                           requires_grad=False)
7
8     def forward(self, hidden_states, labels, attention_mask=None, tau=1.5):
9         # Pool only supervised tokens; fall back to attention_mask if needed.
10        mask = supervised_mask(labels, attention_mask) # [B, T]
11        pooled = masked_mean(hidden_states, mask) # [B, H]
12        routing_scores = self.router(pooled) + self.expert_bias
13        gate_probs = torch.softmax(routing_scores / tau, dim=-1)
14        return gate_probs, routing_scores
```

Listing A.1: Minimal PyTorch-style gate used by GRACE.

A.2 Core Training Step

Listing A.2 keeps only the main training path: student CE, per-teacher ULD, top- k plus capacity routing, GRACE reweighting from cosine alignment, and the final objective.

```
1 def grace_step(student, teachers, batch, gate, state):
2     # 1. Student forward.
3     out_s = student(**batch, return_dict=True)
4     z_s, L_ce = out_s.logits, out_s.loss
5
6     # 2. Student-conditioned gate.
7     h_s = hidden_before_lm_head(student) # [B, T, H]
8     p_gate, s_gate = gate(h_s, batch["labels"],
9                           batch.get("attention_mask"))
```

```

10
11 # 3. Sparse routing with top-k and capacity control.
12 p_route = topk_capacity_route(
13     routing_scores=s_gate,
14     gate_probs=p_gate,
15     top_k=state.top_k,
16     capacity_factor=state.capacity_factor,
17 ) # [B, M]
18
19 # 4. Per-teacher KD loss and alignment score.
20 kd_losses, align_scores = [], []
21 g_ce = pooled_ce_grad(z_s.detach(), batch["labels"]) # [B, V]
22
23 for teacher, teacher_batch in zip(teachers, build_teacher_batches(batch)):
24     z_t, y_t = forward_teacher(teacher, teacher_batch)
25     kd_losses.append(per_sample_uld(z_s, batch["labels"], z_t, y_t))
26
27     g_kd = pooled_kd_grad(z_s.detach(), batch["labels"], z_t, y_t)
28     align_scores.append(cosine_similarity(g_ce, g_kd, dim=-1))
29
30 L_kd_all = torch.stack(kd_losses, dim=-1) # [B, M]
31 S = torch.stack(align_scores, dim=-1) # [B, M]
32
33 # 5. GRACE blends router weights with gradient-alignment weights.
34 W = grace_blend(
35     router_weights=p_route,
36     grace_scores=S,
37     prev_ema=state.grace_score_ema,
38     beta=state.grace_softmax_beta,
39     blend_lambda=state.grace_router_blend_lambda,
40     epsilon=state.grace_epsilon,
41 ) # [B, M]
42
43 # If routed teachers are nearly tied, use uniform weights on the routed set.
44 L_kd = (L_kd_all * W).sum(dim=-1).mean()
45
46 # 6. Balance regularization from soft and hard load.
47 L_bal = gate_balance_loss(p_gate, p_route)
48
49 # 7. Final objective used by the current repository.
50 L = L_ce + (1.0 - state.alpha) * L_kd + state.lambda_bal * L_bal
51 return L

```

Listing A.2: Minimal PyTorch-style GRACE step.

The helpers above correspond to the main repository modules: `teacher_gate.py` for the gate, `routing_utils.py` for top- k and capacity routing, `alignment_utils.py` and `gradient_utils.py` for pooled gradients, `grace.py` for alignment-based blending, and `loss.py` for ULD.

References

- [1] Wikimedia Commons contributors. File: Contrastive language-image pretraining.png. Wikimedia Commons, 2024. MIT License. Accessed: 2026-03-06.
- [2] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, 2021.
- [3] Wikimedia Commons contributors. File: Transformer, attention block diagram.png. Wikimedia Commons, 2024. CC BY 4.0. Accessed: 2026-03-06.
- [4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 5998–6008, 2017.
- [5] Wikimedia Commons contributors. File: Vision transformer.png. Wikimedia Commons, 2024. CC BY 4.0. Accessed: 2026-03-06.
- [6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021.
- [7] Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. Towards vqa models that can read, 2019.
- [8] Minesh Mathew, Dimosthenis Karatzas, and C. V. Jawahar. Docvqa: A dataset for vqa on document images, 2021.
- [9] Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. Chartqa: A benchmark for question answering about charts with visual and logical reasoning, 2022.
- [10] Wikimedia Commons contributors. File: Knowledge-distillation-example.png. Wikimedia Commons, 2023. CC0 1.0. Source: Intel Distiller knowledge distillation page. Accessed: 2026-03-07.
- [11] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015.

- [12] Wikimedia Commons contributors. File: Optimal transport matrix.png. Wikimedia Commons, 2018. CC BY-SA 4.0. Accessed: 2026-03-31.
- [13] Gabriel Peyré and Marco Cuturi. Computational optimal transport, 2019.
- [14] Nicolas Boizard, Kevin El Haddad, Pierre Colombo, and Chloé Clavel. Towards cross-tokenizer distillation: The universal logit distillation loss for llms, 2024.
- [15] Wikimedia Commons contributors. File: Stop sign.jpg. Wikimedia Commons, 2006. Public domain. Accessed: 2026-03-06.
- [16] Wikimedia Commons contributors. File: Receiptswiss.jpg. Wikimedia Commons, 2015. CC BY-SA 3.0. Accessed: 2026-03-06.
- [17] Wikimedia Commons contributors. File: Bar chart.svg. Wikimedia Commons, 2016. CC BY-SA 4.0. Accessed: 2026-03-06.
- [18] Qwen Team. Qwen2-vl-2b-instruct model card. Hugging Face, 2024. Accessed: 2026-04-01.
- [19] Qwen Team. Qwen2.5-vl-3b-instruct model card. Hugging Face, 2025. Accessed: 2026-04-01.
- [20] OpenGVLab. Internvl 3.0. Project blog, 2025. Accessed: 2026-04-01.
- [21] Gemma Team and Google DeepMind. Gemma 3 technical report. Technical report, 2025. Accessed: 2026-04-01.
- [22] Red Hat AI. Qwen2.5-vl-3b-instruct-quantized.w8a8 model card. Hugging Face, 2025. Reports the base-model comparison table including ChartQA. Accessed: 2026-04-01.
- [23] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob Menick, Sebastian Borgeaud, Andrew Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikolaj Binkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karen Simonyan. Flamingo: a visual language model for few-shot learning, 2022.
- [24] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models, 2023.

- [25] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning, 2023.
- [26] Yushi Hu, Otilia Stretcu, Chun-Ta Lu, Krishnamurthy Viswanathan, Kenji Hata, Enming Luo, Ranjay Krishna, and Ariel Fuxman. Visual program distillation: Distilling tools and programmatic reasoning into vision-language models, 2024.
- [27] Xiangxiang Chu, Limeng Qiao, Xinyang Lin, Shuang Xu, Yang Yang, Yiming Hu, Fei Wei, Xinyu Zhang, Bo Zhang, Xiaolin Wei, and Chunhua Shen. Mobilevlm: A fast, strong and open vision language assistant for mobile devices, 2023.
- [28] Baichuan Zhou, Ying Hu, Xi Weng, Junlong Jia, Jie Luo, Xien Liu, Ji Wu, and Lei Huang. Tinyllava: A framework of small-scale large multimodal models, 2024.
- [29] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020.
- [30] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models, 2022.
- [31] Andrés Marafioti, Orr Zohar, Miquel Farré, Merve Noyan, Elie Bakouch, Pedro Cuenca, Cyril Zakka, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Vaibhav Srivastav, Joshua Lochner, Hugo Larcher, Mathieu Morlon, Lewis Tunstall, Leandro von Werra, and Thomas Wolf. Smolvlm: Redefining small and efficient multimodal models, 2025.
- [32] Andrés Marafioti, Merve Noyan, Miquel Farré, Elie Bakouch, and Pedro Cuenca. Smolvlm - small yet mighty vision language model. Hugging Face Blog, 2024. Accessed: 2026-04-02.
- [33] AI Rank. Textvqa benchmark: Complete leaderboard and performance analysis. AI Rank, 2026. Self-reported leaderboard. Accessed: 2026-04-02.
- [34] AI Rank. Docvqa benchmark: Complete leaderboard and performance analysis. AI Rank, 2026. Self-reported leaderboard. Accessed: 2026-04-02.

- [35] AI Rank. Chartqa benchmark: Complete leaderboard and performance analysis. AI Rank, 2026. Self-reported leaderboard. Accessed: 2026-04-02.
- [36] Xiaohan Xu, Ming Li, Chongyang Tao, Tao Shen, Reynold Cheng, Jinyang Li, Can Xu, Dacheng Tao, and Tianyi Zhou. A survey on knowledge distillation of large language models, 2024.
- [37] Sayantan Dasgupta and Trevor Cohn. Improving language model distillation through hidden state matching. In *International Conference on Learning Representations*, 2025.
- [38] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers, 2022.
- [39] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning*, pages 38087–38099, 2023.
- [40] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration, 2024.
- [41] Shiyao Li, Yingchun Hu, Xuefei Ning, Xihui Liu, Ke Hong, Xiaotao Jia, Xiuhong Li, Yaqi Yan, Pei Ran, Guohao Dai, Shengen Yan, Huazhong Yang, and Yu Wang. Mbq: Modality-balanced quantization for large vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4167–4177, 2025.
- [42] Kaicheng Yang, Tiancheng Gu, Xiang An, Haiqiang Jiang, Xiangzi Dai, Ziyong Feng, Weidong Cai, and Jiankang Deng. Clip-cid: Efficient clip distillation via cluster-instance discrimination, 2024.
- [43] Fei Yuan, Linjun Shou, Jian Pei, Wutao Lin, Ming Gong, Yan Fu, and Daxin Jiang. Reinforced multi-teacher selection for knowledge distillation, 2020.
- [44] Hailin Zhang, Defang Chen, and Can Wang. Adaptive multi-teacher knowledge distillation with meta-learning, 2023.
- [45] Yu-Chu Yu, Chi-Pin Huang, Jr-Jen Chen, Kai-Po Chang, Yung-Hsuan Lai, Fu-En Yang, and Yu-Chiang Frank Wang. Select and distill: Selective dual-teacher knowledge transfer for continual learning on vision-language models, 2024.

- [46] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, 2022.
- [47] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew Dai, Zhifeng Chen, Quoc Le, and James Laudon. Mixture-of-experts with expert choice routing, 2022.
- [48] Yichen Zhu and Yi Wang. Student customized knowledge distillation: Bridging the gap between student and teacher. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 5057–5066, 2021.
- [49] Jianhua Lin. Divergence measures based on the shannon entropy. *IEEE Transactions on Information Theory*, 37(1), 1991.
- [50] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, 2017.
- [51] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024.
- [52] Wikimedia Commons contributors. File: Deepseek moe and mla (deepseek-v2).svg. Wikimedia Commons, 2025. MIT License. Accessed: 2026-03-06.
- [53] Haodong Duan, Xinyu Fang, Junming Yang, et al. Vlmevalkit: An open-source toolkit for evaluating large multi-modality models, 2024.
- [54] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models, 2020.
- [55] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost, 2016.
- [56] Cloudflare. R2 object storage. Cloudflare developer documentation, 2026. Accessed: 2026-04-09.
- [57] Cloudflare. Use the aws sdk for python (boto3) with r2. Cloudflare developer documentation, 2026. Accessed: 2026-04-09.
- [58] fsspec developers. Features. fsspec documentation, 2026. Accessed: 2026-04-09.
- [59] fsspec developers. fsspec.implementations.cached. fsspec documentation, 2026. Accessed: 2026-04-09.

- [60] Hugging Face TB. Smolvlm2-2.2b-instruct model card. Hugging Face, 2025. Accessed: 2026-04-02.
- [61] Yifan Cui, Wei Wang, Yifei Wang, Yao Lu, Haoyu Wang, Fuzhen Zhuang, Qing He, and Chuan Shi. Sinkhorn distance minimization for knowledge distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 11046–11060. Association for Computational Linguistics, 2024.
- [62] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3):229–256, 1992.
- [63] Yuqiao Wen, Zichao Li, Wenyu Du, and Lili Mou. f-divergence minimization for sequence-level knowledge distillation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10817–10834. Association for Computational Linguistics, 2023.
- [64] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3519–3529. PMLR, 2019.